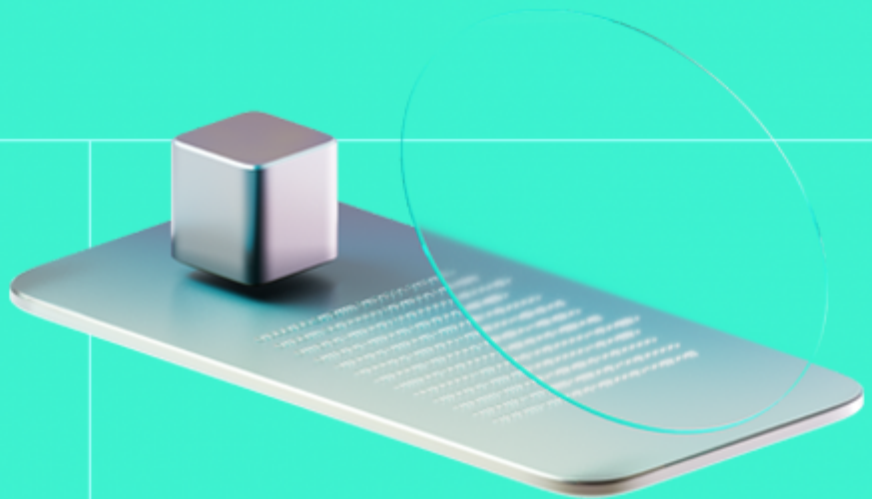




Smart Contract Code Review And Security Analysis Report

Customer: Push Chain

Date: 05/06/2026



We express our gratitude to the Push Chain team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Push Chain Gateway is a cross-chain interoperability protocol that bridges funds and payloads between external EVM chains and Push Chain. The protocol uses a TSS (Threshold Signature Scheme) relay network to finalize transactions across chains.

Document

Name	Smart Contract Code Review and Security Analysis Report for Push Chain
Audited By	Olesia Bilenka; Kornel Światłowski
Approved By	Khrystyna Tkachuk
Website	https://push.org
Changelog	08/04/2026 - Preliminary Report
	14/05/2026 - Final Report
	02/06/2026 - Post Audit Changes
Platform	Push Chain, Ethereum
Language	Solidity
Tags	Bridge; Centralization; Upgradable; Vault; Signatures
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/pushchain/push-chain-gateway-contracts
Commit	0928967
Remediation commit	4249311
Post Audit Changes	80f7a20

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

38

Total Findings

28

Resolved

6

Accepted

4

Mitigated

Findings by Severity

Severity

Count

Critical

0

High

2

Medium

6

Low

10

Vulnerability	Severity	Status
F-2026-15648 - UniversalGatewayPC Relies on IUniversalCore.BASE_GAS_LIMIT(), but UniversalCore Does Not Implement It	High	Fixed
F-2026-15737 - Fee-on-Transfer ERC20 Tokens Cause Vault Insolvency and Permanent Fund Lock in Revert/Rescue Paths	High	Fixed
F-2026-15642 - Inconsistent Role Management Causes State and Role Mismatch	Medium	Fixed
F-2026-15643 - updateEpochDuration Implicitly Resets Rate-Limit Usage for All Tokens	Medium	Mitigated
F-2026-15646 - Documentation and Implementation Mismatch in Token Support Validation	Medium	Fixed
F-2026-15656 - _burnPRC20 Ignores transferFrom Return Value, Allowing Silent Transfer Failure	Medium	Fixed
F-2026-15658 - rescueFundsOnSourceChain Lacks Replay Protection and universalTxId-Token Binding	Medium	Accepted
F-2026-15731 - Inconsistent Recipient Semantics Between Documentation and Funds Event Emission	Medium	Accepted
F-2026-15639 - Absence of Granular Role Separation	Low	Fixed
F-2026-15655 - Missing Rescue Gas Limit Controls in UniversalGatewayPC Implementation	Low	Fixed
F-2026-15657 - UniversalGatewayPC Lacks an Admin Setter for UNIVERSAL_CORE	Low	Fixed
F-2026-15659 - Users Have No Control Over Swap Parameters in Outbound Gas-Fee Conversion	Low	Mitigated

Vulnerability	Severity	Status
F-2026-15660 - setVault Migration Can Orphan Previously Locked ERC20 Funds in the Old Vault	Low	Mitigated
F-2026-15683 - msg.value Is Permanently Trapped in sendUniversalTx(UniversalTokenTxRequest)	Low	Fixed
F-2026-15684 - Transaction Type Is Inferred Before Fee Deduction, Causing Post-Fee Route Mismatch	Low	Fixed
F-2026-15686 - Native FUNDS Route Does Not Support Gas Top-Up Batching	Low	Accepted
F-2026-15740 - Refund to Non-Receiving Contract Can Revert and Block Entire Outbound Transaction	Low	Accepted
F-2026-15766 - Misleading TSS Control Description in UniversalGateway Documentation	Low	Fixed
F-2026-15636 - Missing Event Emissions for Critical Admin Configuration Changes	Info	Accepted
F-2026-15638 - Redundant TSS_ROLE Management (Role Not Used for Authorization)	Info	Fixed
F-2026-15640 - Admin Front-Run Risk on Protocol Fee Updates	Info	Mitigated
F-2026-15641 - Unbounded setProtocolFee Allows Arbitrary Fee Configuration	Info	Fixed
F-2026-15645 - setChainlinkStalePeriod(0) Disables Oracle Freshness Validation	Info	Fixed
F-2026-15682 - Misleading Function Name setRouters	Info	Fixed
F-2026-15693 - Per-Block USD Cap Uses Mixed-Price Accounting Within the Same Block	Info	Fixed
F-2026-15725 - Redundant Initializable and ContextUpgradeable Inheritance	Info	Fixed
F-2026-15733 - ETH Permanently Locked in Gateway on ERC20 Revert/Rescue Due to Missing msg.value Validation	Info	Fixed
F-2026-15734 - Inconsistent State Variable Ordering in UniversalGatewayPC Reduces Upgradeability Clarity	Info	Fixed
F-2026-15735 - Mutable State Variables Are Named Like Constants	Info	Fixed
F-2026-15736 - Dead WETH Branch in _findV3PoolWithNative	Info	Fixed
F-2026-15738 - initialize Silently Skips Uniswap Configuration If Only One Address Is Zero	Info	Fixed
F-2026-15739 - VaultPC Withdraw Events Omit Recipient Address	Info	Fixed
F-2026-15741 - Redundant Deadline Validation in Token-Gas Flow	Info	Fixed
F-2026-15742 - Inconsistent Function Naming vs Visibility Convention	Info	Fixed
F-2026-15743 - Unreachable else Branch in _routeUniversalTx	Info	Fixed
F-2026-15744 - Duplicate Logic in revertUniversalTx and rescueFunds Increases Maintenance Risk	Info	Accepted
F-2026-15745 - Redundant else if Branch in _sendTxWithFunds	Info	Fixed

Vulnerability	Severity	Status
F-2026-15746 - IUniversalGateway.UniversalTxExecuted Is Declared but Never Emitted	Info	Fixed

Documentation quality

- The project provides comprehensive documentation covering architecture, transaction flows, and security analysis. High-level architecture is described in **1_PUSH_CHAIN.md** with clear actor definitions and component relationships.
- Per-contract documentation (**2_UniversalGateway.md**, **3_UniversalGatewayPC.md**) includes TX_TYPE inference tables, rate-limit mechanics, fee models with worked examples, and sequence diagrams. Full outbound and inbound transaction flow documentation is provided in **4_OutboundTx_Flows.md** (covering all four categories: withdrawals, DeFi execution, CEA self-calls, and reverts with Mermaid sequence diagrams for each variant) and **5_InboundTx_Flows.md**. Revert handling, threat modelling, and upgrade plans are documented separately.
- NatSpec comments are present on most public and internal functions. Minor inconsistencies exist: the **IUniversalCore** interface in the gateway declares **BASE_GAS_LIMIT()** which does not exist on the actual **UniversalCore** implementation, and some line references in documentation do not match current code after remediation commits.

Code quality

- The codebase properly reuses OpenZeppelin upgradeable libraries for access control, pausability, reentrancy protection, and safe ERC-20 operations.
- Code duplication exists in both **UniversalGateway** and **Vault**, where identical transfer logic is copy-pasted with only the emitted event name differing.
- Function naming and visibility conventions are inconsistent. Mutable state variables use CONSTANT-style **ALL_CAPS** naming across all contracts.
- The development environment is fully configured with Foundry, including profiles for CI, coverage, debugging, and fork testing.

Test coverage

Code coverage of the project is **76.61%** (branch coverage).

- Tests cover admin actions, all four transaction types (**GAS**, **GAS_AND_PAYLOAD**, **FUNDS**, **FUNDS_AND_PAYLOAD**), rate limiting (block-based and epoch-based), oracle integration, protocol fees, CEA-based inbound flows (58 tests), rescue operations, vault withdrawals, and vault token migration.
- Fuzz tests are provided for protocol fee collection, rate limits, transaction routing, and vault invariants. The primary coverage gap is in **UniversalGateway.sol** branch coverage (70%) — uncovered branches include edge cases in Uniswap v3 pool discovery, concurrent multi-user rate-limit exhaustion, and certain L2 sequencer oracle failure paths.

Table of Contents

System Overview	8
Privileged Roles	8
Potential Risks	10
Findings	12
Vulnerability Details	12
Disclaimers	104
Appendix 1. Definitions	105
Severities	105
Potential Risks	105
Appendix 2. Scope	106
Appendix 3. Additional Valuables	107

System Overview

Push Chain Gateway is a cross-chain interoperability protocol that bridges funds and payloads between external EVM chains and Push Chain. The protocol uses a TSS (Threshold Signature Scheme) relay network to finalize transactions across chains. The following contracts are in scope:

- **UniversalGateway** — the primary inbound gateway deployed on external EVM chains (e.g., Ethereum, Arbitrum). It accepts user deposits of native ETH or whitelisted ERC-20 tokens and routes them to Push Chain. Supports four transaction types: pure gas top-ups, gas with payload execution, large-ticket fund transfers, and fund transfers with payload. Enforces per-transaction USD caps and per-block USD caps (via Chainlink ETH/USD oracle) for the instant gas route, and epoch-based per-token rate limits for the standard fund route. Supports ERC-20-to-native gas abstraction via Uniswap V3 swaps. Collects a flat native protocol fee on each inbound transaction. Provides revert and rescue paths for failed cross-chain transactions, callable only by the **Vault**. Uses OpenZeppelin UUPS-compatible upgradeability, access control (admin, TSS, Vault, pauser roles), pausability, and reentrancy guards.
- **Vault** — the ERC-20 custody contract on external EVM chains. Holds bridged tokens and releases them on the destination chain upon TSS instruction. Routes all outbound executions through deterministic Chain Execution Accounts (CEAs) deployed via a factory. Supports three outbound operations: finalize (withdraw or execute via CEA), revert (refund failed inbound transactions), and rescue (recover stuck funds). All state-changing operations are restricted to the TSS role. Uses OpenZeppelin upgradeability, access control, pausability, and reentrancy guards.
- **UniversalGatewayPC** — the outbound gateway deployed on Push Chain. Handles user-initiated transactions from Push Chain to external EVM chains. Burns PRC-20 synthetic tokens at request time, collects a protocol fee in native PC sent to **VaultPC**, and swaps remaining native PC to gas tokens via **UniversalCore** for cross-chain execution costs. Supports three outbound transaction types: funds only, funds with payload, and payload with gas. Also provides a rescue-funds flow for requesting release of stuck tokens on external chains. Uses OpenZeppelin upgradeability, access control, pausability, and reentrancy guards.
- **VaultPC** — a simple fee custody vault deployed on Push Chain. Stores protocol fees collected from outbound transactions. Supports withdrawal of both native PC and PRC-20 tokens by a designated manager role. Uses OpenZeppelin upgradeability, access control, pausability, and reentrancy guards.
- **Types** — shared library defining the transaction type enum (GAS, GAS_AND_PAYLOAD, FUNDS, FUNDS_AND_PAYLOAD, RESCUE_FUNDS), revert instruction struct, multicall struct for CEA execution, packed epoch-usage struct for rate limiting, and signature verification types.
- **TypesUG** — defines the inbound request structs for **UniversalGateway**: a standard request (native gas) and a token-gas request (ERC-20 gas with Uniswap swap parameters).
- **TypesUGPC** — defines the outbound request struct for **UniversalGatewayPC**, including recipient (raw bytes for cross-VM compatibility), PRC-20 token, amount, gas limit, payload, and revert recipient.
- **Errors** — shared custom error library used across all gateway and vault contracts. Defines common validation errors and domain-specific errors for deposits, withdrawals, rate limiting, slippage, and token burns.

Privileged roles

- **The DEFAULT_ADMIN_ROLE holder in the UniversalGateway contract** can update all critical protocol parameters including the TSS address (via `updateTSS`), the Vault address (via `updateVault`, which also transfers VAULT_ROLE), USD rate-limit caps, the per-block USD cap, Uniswap V3 router and factory addresses, Chainlink oracle feed addresses and staleness thresholds, L2 sequencer feed and grace period, the CEA factory address, the protocol fee amount, token whitelist and rate-limit thresholds, and the epoch duration. Changing these parameters takes effect immediately without a timelock or delay.
- **The DEFAULT_ADMIN_ROLE holder in the Vault contract** can update the gateway address (via `setGateway`), the CEA factory address (via `setCEAFactory`), and migrate all token balances (ERC20 and native) to a new vault (via `migrateTokens`, which requires both the Vault and Gateway to be paused). These changes take effect immediately.
- **The DEFAULT_ADMIN_ROLE holder in the UniversalGatewayPC contract** can update the VaultPC address (via `setVaultPC`) and the UniversalCore address (via `setUniversalCore`). These changes take effect immediately.
- **The DEFAULT_ADMIN_ROLE holder in the VaultPC contract** has no protocol-specific configuration setters beyond granting and revoking other roles.
- **Across all four contracts**, the DEFAULT_ADMIN_ROLE holder can grant and revoke any other role, including granting the admin role itself to other addresses.
- **The TSS_ROLE holder in the Vault contract** is the sole authority that can finalize outbound transactions (releasing funds to users on external chains via CEA contracts), revert failed inbound transactions (returning funds to the original sender via the Gateway), and rescue stuck funds. All fund movement from the Vault to end users passes through this role.
- **Note:** TSS_ROLE is **not defined** in the UniversalGateway contract. The Gateway tracks a `TSS_ADDRESS` state variable that serves as the recipient for native fees and deposits, but TSS-related authorization for outbound operations is enforced exclusively in the Vault contract. The previous TSS_ROLE in UniversalGateway was removed as it was not used for any access control check.
- **The VAULT_ROLE holder in the UniversalGateway contract** is the only address authorized to trigger revert and rescue operations on the gateway. In practice, this role is assigned to the Vault contract, allowing it to route refund and rescue flows through the gateway's replay-protected transfer logic.
- **The PAUSER_ROLE holder** can pause and unpause the respective contract. When paused, all user-facing operations (deposits, outbound transactions, finalizations, reverts, rescues) are halted. This role exists in all four contracts: UniversalGateway, Vault, UniversalGatewayPC, and VaultPC.
- **The MANAGER_ROLE holder in the VaultPC contract** can withdraw any amount of native PC tokens or PRC-20 tokens from the fee vault to any address. This role has full custody authority over all fees accumulated in VaultPC.

Potential Risks

- **Centralized Admin Control Without Timelock:** The admin role holder across **UniversalGateway**, **Vault**, **UniversalGatewayPC**, and **VaultPC** can modify critical protocol parameters with immediate effect, including TSS and Vault references, oracle feeds, Uniswap configuration, CEA factory, rate limits, protocol fees, and token support settings. In **Vault**, the admin can also migrate all ERC-20 and native balances to a new vault while the system is paused. No timelock, governance vote, or multi-step confirmation is required.
- **TSS Single Point of Trust:** The TSS authority is the most privileged operational actor. It controls outbound finalization through **Vault**, token reverts, rescues, and indirectly native reverts/rescues through **UniversalGateway**. If compromised or unavailable, TSS can drain Vault-held assets, retain deposited native funds, suppress relaying, or leave users without an on-chain recovery path.
- **Off-Chain Relay Liveness Dependency:** Cross-chain execution depends on TSS observing events and submitting destination-chain transactions. Sustained TSS downtime, RPC failures, event censorship, or network partitions can leave ERC-20 tokens locked in **Vault**, native funds held by TSS, or PRC-20 tokens burned on Push Chain without corresponding release. The rescue path in **UniversalGatewayPC** still depends on TSS execution.
- **Manager Role Has Unrestricted Fee Withdrawal:** The manager role holder in **VaultPC** can withdraw all native PC and PRC-20 balances to any address without per-withdrawal limits, cooldowns, or multi-signature confirmation. If the role is assigned to a single compromised EOA, all accumulated protocol fees are immediately drainable.
- **Pauser Role Can Halt All Operations Indefinitely:** The pauser role holder in **UniversalGateway**, **Vault**, **UniversalGatewayPC**, and **VaultPC** can halt user-facing operations indefinitely, including deposits, outbound transactions, finalizations, reverts, and rescues. There is no forced-unpause mechanism or automatic expiry, and recovery requires role reassignment if the pauser key is lost or compromised.
- **Oracle Dependency for Rate Limiting:** Instant-route USD caps depend on the Chainlink ETH/USD feed. If the oracle returns stale, unavailable, or manipulated prices, the protocol may enforce incorrect real-value limits. Existing staleness and L2 sequencer checks reduce this risk but do not provide a fallback oracle or full circuit-breaker mechanism.
- **Upgradeable Contracts Without Timelock:** **UniversalGateway**, **Vault**, **UniversalGatewayPC**, and **VaultPC** use the OpenZeppelin upgradeable proxy pattern. The proxy admin can upgrade contract logic immediately without a mandatory review period, timelock, or governance approval, creating risk of malicious or flawed upgrades.
- **Interactions with External DeFi Protocols:** The system relies on Uniswap v3 for gas-related swaps and allows CEA multicall execution against arbitrary external DeFi protocols. This exposes users and protocol flows to inherited risks such as pool manipulation, router issues, liquidity depletion, and vulnerabilities in target protocols.
- **Report Modification Notice:** This report was modified on May 27th, 2026, at the client's request to incorporate minor code changes introduced up to commit [80f7a20](#). While these updates were intended to align the report with the latest version of the codebase provided by the client, modifications to previously published audit content may affect the continuity and integrity of the original findings. Hacken reviewed the updated changes to confirm that they correspond solely to the requested modifications. However, any future changes involving substantial code updates or

additional commits should undergo a separate security assessment to ensure that no new risks are introduced to the project's security posture.

Findings

Vulnerability Details

[F-2026-15648](#) - UniversalGatewayPC Relies on IUniversalCore.BASE_GAS_LIMIT(), but UniversalCore Does Not Implement It - High

Description: **UniversalGatewayPC** relies on `IUniversalCore.BASE_GAS_LIMIT` when the user does not explicitly provide a gas limit:

```
gasLimitUsed = gasLimit == 0 ? IUniversalCore(UNIVERSAL_CORE).BASE_GAS_LIMIT(
) : gasLimit;

(gasToken, gasFee, protocolFee, gasPrice, chainNamespace) =
    IUniversalCore(UNIVERSAL_CORE).getOutboundTxGasAndFees(token, gasLimitUse
d);
```

The interface explicitly declares `BASE_GAS_LIMIT`:

```
interface IUniversalCore {
    /**
     * @notice Get base gas limit for a chain
     * @return baseGasLimit Base gas limit
     */
    function BASE_GAS_LIMIT() external view returns (uint256 baseGasLimit);
```

However, the provided **UniversalCore** implementation does not define either:

- a `BASE_GAS_LIMIT` function, or
- a public state variable named `BASE_GAS_LIMIT`.

Instead, it uses a per-chain mapping:

```
function getOutboundTxGasAndFees(address _prc20, uint256 gasLimitWithBaseLimi
t)
    public
    view
    returns (address gasToken, uint256 gasFee, uint256 protocolFee, uint256 ga
sPrice, string memory chainNamespace)
{
    chainNamespace = IPRC20(_prc20).SOURCE_CHAIN_NAMESPACE();
```

```

uint256 baseLimit = baseGasLimitByChainNamespace[chainNamespace];
if (gasLimitWithBaseLimit == 0) {
    gasLimitWithBaseLimit = baseLimit;
} else if (gasLimitWithBaseLimit < baseLimit) {
    revert UniversalCoreErrors.GasLimitBelowBase(gasLimitWithBaseLimit, baseLimit);
}
// ...
}

```

This creates an interface/implementation mismatch with the following consequences:

- The `gasLimit == 0` path in **UniversalGatewayPC** will call a non-existent selector on **UniversalCore**, causing the transaction to revert.
- As a result, all outbound transactions that rely on the default gas-limit flow may become unusable.

Assets:

- UniversalGatewayPC.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations

Remediation:

Align the interface and implementation. Any of the following would fix the issue:

- Remove the `BASE_GAS_LIMIT` call from **UniversalGatewayPC** and pass `gasLimit` directly into `getOutboundTxGasAndFees`, since **UniversalCore** already handles `0` as "use default".
- Or add a real `BASE_GAS_LIMIT` implementation if a global default is actually intended.
- Or update `IUniversalCore` to reflect the real per-chain model and avoid exposing a non-existent global getter.

Resolution:

Fixed in [56c5372](#) (evm-gateway) and [51843b3](#) (universal-core):

Commits [56c5372](#) in [evm-gateway](#) and commit [51843b3](#) in **UniversalCore** jointly resolve the interface/implementation mismatch between **UniversalGatewayPC** and **IUniversalCore** regarding gas limit resolution.

The fix addresses the identified concern through coordinated changes across both repositories:

- **Removal of non-existent interface declaration:** The `BASE_GAS_LIMIT` function declaration has been removed from **IUniversalCore**. The interface no longer references a function that does not exist in **UniversalCore**.
- **Delegation of gas limit resolution:** Gas limit defaulting logic has been moved entirely to **UniversalCore**. When `gasLimit = 0` is passed to `getOutboundTxGasAndFees`, **UniversalCore** internally resolves it to the per-chain `baseGasLimitByChainNamespace[chainNamespace]` value.
- **Extended return signature:** `getOutboundTxGasAndFees` now returns `gasLimitUsed` as a sixth return value, allowing **UniversalGatewayPC** to receive the effective gas limit without requiring a separate external call.

Evidences

PoC: Default Gas Limit Reverts

Reproduce:

```
function test_DefaultGasLimit_Reverts() public {  
    // Calling with gasLimit = 0 triggers BASE_GAS_LIMIT() call  
    // MockUniversalCore has no BASE_GAS_LIMIT() → reverts  
    vm.expectRevert();  
    gateway.tryOutboundDefaultGas(address(prc20));  
}
```

Results:

```
[PASS] test_DefaultGasLimit_Reverts() (gas: 15540)
```

[F-2026-15737](#) - Fee-on-Transfer ERC20 Tokens Cause Vault Insolvency and Permanent Fund Lock in Revert/Rescue Paths - High

Description:

`_handleDeposits` from **UniversalGateway** transfers **ERC20** tokens from the user directly to the **Vault** using the user-declared amount without verifying how many tokens the **Vault** actually received:

```
IERC20(token).safeTransferFrom(_msgSender(), VAULT, amount);
```

No `balanceOf` before/after check is performed. If a whitelisted token implements fee-on-transfer (transfer tax), the **Vault** receives fewer tokens than recorded. The same nominal amount is used for:

1. Rate-limit accounting — `_consumeRateLimit(token, _req.amount)` records the full declared amount.
2. Event emission — `UniversalTx` event logs the full `_req.amount`, which Push Chain uses to credit the user.
3. **Vault** balance — actually receives `amount - transferFee`.

This creates a progressive deficit: Push Chain believes the **Vault** holds more than it actually does.

The issue compounds across the revert/rescue path. When `revertUniversalTx` or `rescueFunds` processes an **ERC20** refund, it performs a double transfer:

```
IERC20(token).safeTransfer(address(gateway), amount);
gateway.revertUniversalTx(
    subTxId, universalTxId, token, amount, revertInstruction
);
IERC20(token).safeTransfer(revertInstruction.revertRecipient, amount);
```

For a fee-on-transfer token: **Vault** sends amount, **Gateway** receives `amount - fee`, **Gateway** attempts to forward amount to the recipient — the transfer reverts due to insufficient balance. This makes all ERC20 reverts and rescues for fee-on-transfer tokens permanently impossible.

Additionally, `swapToNative` is also affected:

```
IERC20(tokenIn).safeTransferFrom(_msgSender(), address(this), amountIn);
IERC20(tokenIn).safeIncreaseAllowance(address(uniV3Router), amountIn);
```

The **Gateway** receives `amountIn - fee` but approves `amountIn` to the router. When the router attempts to pull the full `amountIn`, the call reverts. All token-as-gas swaps for fee-on-transfer tokens are broken.

Three distinct failure modes from a single root cause:

Flow	Impact
Inbound deposits	Vault receives less than recorded → progressive insolvency; eventually unable to honor withdrawals — direct loss of funds
Revert / rescue	Gateway receives less than it tries to forward → permanent revert; user funds irrecoverable with no alternative recovery path
Token-as-gas swap	Uniswap router pull exceeds actual balance → permanent revert;

The protocol whitelists **ERC20** tokens via `setTokenLimitThresholds` with no restriction or validation against fee-on-transfer behavior.

Once a fee-on-transfer token is active, the insolvency accumulates silently with every deposit, and the revert/rescue path is permanently broken with no on-chain recovery mechanism available in the **Gateway**.

Client explicitly requested verification of fee-on-transfer behavior in the protocol fee collection flow ("`If a supported ERC-20 has fee-on-transfer mechanics, confirm the gateway accounts for the reduced amount received`"). This requirement materially affects severity assessment: fee-on-transfer is not an out-of-scope edge case but a declared validation criterion.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]
- Vault.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate:	5/5
Likelihood Rate:	5/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	High

Recommendations

Remediation: Add a balance-before/after check in `_handleDeposits` to reject tokens where the received amount differs from the declared amount:


```
function _handleDeposits(address token, uint256 amount) internal {
    if (token == address(0)) {
        (bool ok,) = payable(TSS_ADDRESS).call{ value: amount }("");
        if (!ok) revert Errors.DepositFailed();
    } else {
        if (tokenToLimitThreshold[token] == 0) revert Errors.NotSupported();
        uint256 balBefore = IERC20(token).balanceOf(VAULT);
        IERC20(token).safeTransferFrom(_msgSender(), VAULT, amount);
        uint256 received = IERC20(token).balanceOf(VAULT) - balBefore;
        if (received != amount) revert Errors.InvalidAmount();
    }
}
```

Apply the same pattern in:

- `_finalizeUniversalTx` from **Vault** for outbound ERC20 transfers to CEA.
- `swapToNative` from **UniversalGateway** for the `safeTransferFrom` to the Gateway.

Resolution:

Fixed in `f31052c`:

`handleDeposits` now measures `IERC20(token).balanceOf(VAULT)` before and after `safeTransferFrom` and reverts with `Errors.InvalidAmount` unless the Vault receives the full declared amount. `_swapToNative` now applies the same exact-receipt validation before router approval. Fee-on-transfer tokens are therefore rejected at ingress, preventing Vault accounting drift, preventing broken token-as-gas swaps, and preventing fee-on-transfer assets from entering the revert and rescue flow through supported deposits. Regression coverage was added through `MockFeeOnTransferERC20` and `test_SendTxWithFunds_FeeOnTransferERC20_Reverts`.

Evidences

PoC: Fee-on-Transfer ERC20 Tokens Cause Vault Insolvency and Permanent Fund Lock in Revert/Rescue Paths

Reproduce:

```
// =====
// TEST 1: Inbound deposit – Vault receives less than recorded
// =====
function test_InboundDeposit_VaultUnderfunded() public {
    uint256 vaultBefore = token.balanceOf(address(vault));
    assertEq(vaultBefore, 0, "Vault starts empty");

    // Gateway records DEPOSIT_AMOUNT but Vault receives less due to 2% f
```

ee

```
vm.prank(user);
gateway.handleDeposit(token, DEPOSIT_AMOUNT, user);

uint256 vaultAfter = token.balanceOf(address(vault));
uint256 expectedFee = (DEPOSIT_AMOUNT * 200) / 10_000; // 2%
uint256 expectedReceived = DEPOSIT_AMOUNT - expectedFee;

// Vault received LESS than the declared amount
assertEq(vaultAfter, expectedReceived, "Vault receives amount minus t
ransfer fee");
assertTrue(vaultAfter < DEPOSIT_AMOUNT, "CRITICAL: Vault balance < de
clared amount");

uint256 deficit = DEPOSIT_AMOUNT - vaultAfter;
assertEq(deficit, expectedFee, "Deficit equals the transfer fee");
}

// =====
// TEST 2: Progressive insolvency after multiple deposits
// =====
function test_ProgressiveInsolvency() public {
    uint256 totalDeclared = 0;

    // Simulate 10 deposits of 100 tokens each
    for (uint256 i = 0; i < 10; i++) {
        vm.prank(user);
        gateway.handleDeposit(token, DEPOSIT_AMOUNT, user);
        totalDeclared += DEPOSIT_AMOUNT;
    }

    uint256 vaultBalance = token.balanceOf(address(vault));
    uint256 totalDeficit = totalDeclared - vaultBalance;

    // After 10 deposits of 100 tokens (2% fee each):
    // Declared: 1000 tokens
    // Actual: 980 tokens
    // Deficit: 20 tokens
    assertTrue(vaultBalance < totalDeclared, "INSOLVENCY: Vault holds les
s than total declared");
    assertEq(vaultBalance, 980 ether, "Vault has 980 tokens (2% lost per
deposit)");
}
```

```

    assertEq(totalDeficit, 20 ether, "20 tokens lost to transfer fees");
}

// =====
// TEST 3: Revert path – Gateway receives less than it
//          needs to forward → permanently broken
// =====
function test_RevertPath_GatewayInsufficientBalance() public {
    // Simulate: Vault has 100 tokens
    token.mint(address(vault), 100 ether);

    // Vault sends 100 to Gateway → Gateway receives 98 (2% fee)
    // Gateway tries to forward 100 to recipient → REVERTS

    vm.expectRevert("balance");
    vault.revertERC20(token, 100 ether, recipient);

    // After revert: everything rolled back. Tokens still in Vault.
    // But the user can NEVER get their funds back via this path.
    assertEq(token.balanceOf(address(vault)), 100 ether, "Tokens stuck in
Vault after failed revert");
    assertEq(token.balanceOf(recipient), 0, "Recipient received nothing")
;
}

// =====
// TEST 4: Demonstrates the double-fee drain numerically
// =====
function test_DoubleFee_NumericTrace() public {
    uint256 originalAmount = 100 ether;

    // Mint to Vault
    token.mint(address(vault), originalAmount);

    // Trace what happens step by step:
    // Step 1: Vault.transfer(gateway, 100) → fee = 2% of 100 = 2
    //          Gateway receives 98
    // Step 2: Gateway.transfer(recipient, 100) → needs 100 but has 98
    //          REVERTS with "balance"

    // Verify step 1 numerically

```

```

vm.prank(address(vault));
token.transfer(address(gateway), originalAmount);

uint256 gatewayBal = token.balanceOf(address(gateway));
uint256 expectedGateway = originalAmount - (originalAmount * 200 / 10
_000); // 98

assertEq(gatewayBal, expectedGateway, "Gateway received 98 after 2% fee");

// Step 2: Gateway tries to forward 100 but only has 98
vm.expectRevert("balance");
vm.prank(address(gateway));
token.transfer(recipient, originalAmount);
}

```

Results:

[PASS] test_InboundDeposit_VaultUnderfunded()	(gas: 91709)
[PASS] test_ProgressiveInsolvency()	(gas: 162084)
[PASS] test_RevertPath_GatewayInsufficientBalance()	(gas: 88083)
[PASS] test_DoubleFee_NumericTrace()	(gas: 82504)

[F-2026-15642](#) - Inconsistent Role Management Causes State and Role Mismatch - Medium

Description:

The `Vault` and `UniversalGateway` contracts expose functions such as `setTSS()` and `setVault()`, suggesting a canonical and potentially exclusive mechanism for configuring `TSS_ROLE` and `VAULT_ROLE`.

```
// UniversalGateway
function setTSS(address newTSS) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (newTSS == address(0)) revert Errors.ZeroAddress();
    address old = TSS_ADDRESS;

    // transfer role
    if (hasRole(TSS_ROLE, old)) _revokeRole(TSS_ROLE, old);
    _grantRole(TSS_ROLE, newTSS);

    TSS_ADDRESS = newTSS;
}

function setVault(address newVault) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (newVault == address(0)) revert Errors.ZeroAddress();
    address old = VAULT;

    // transfer role
    if (hasRole(VAULT_ROLE, old)) _revokeRole(VAULT_ROLE, old);
    _grantRole(VAULT_ROLE, newVault);

    VAULT = newVault;
    emit VaultUpdated(old, newVault);
}
```

```
// Vault
function setTSS(address newTss) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (newTss == address(0)) revert Errors.ZeroAddress();
    address old = TSS_ADDRESS;

    if (hasRole(TSS_ROLE, old)) _revokeRole(TSS_ROLE, old);
    _grantRole(TSS_ROLE, newTss);

    TSS_ADDRESS = newTss;
    emit TSSUpdated(old, newTss);
}
```

However, since the contracts inherit from `AccessControlUpgradeable`, role management can also be performed directly through the generic `grantRole()` and `revokeRole()` functions. As a result, multiple configuration paths exist for assigning and revoking roles.

This design introduces ambiguity, as the dedicated setter functions suggest a primary or exclusive configuration mechanism, while role assignments may also occur through inherited access-control functions.

Additionally, during contract initialization and within the setter functions, the address granted a role is stored in corresponding state variables such as `TSS_ADDRESS` or `VAULT`. When only setter functions are used, consistency between the stored address and role assignment is maintained.

However, direct usage of `grantRole` allows multiple addresses to hold the same role, while only a single address is tracked in storage. This results in divergence between role assignments and stored state. Similarly, invoking `revokeRole` may remove a role without updating the corresponding storage variable, leading to stale or invalid state references.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]
- Vault.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	3/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: It is recommended to enforce a single source of truth for role management by restricting direct usage of `grantRole` and `revokeRole`. Alternatively, storage variables should be removed, and role membership should be treated as the authoritative source of access-control state.

Resolution: Fixed in `43e8cc2` :

Commit `43e8cc2` introduces granular role separation with

AccessControlDefaultAdminRulesUpgradeable across all four audited contracts (**UniversalGateway**, **Vault**, **VaultPC**, **UniversalGatewayPC**).

The implementation addresses the identified concern through:

- Admin Transfer Delay: All contracts initialize **AccessControlDefaultAdminRulesUpgradeable** with a 1-day delay (`__AccessControlDefaultAdminRules_init(1 days, admin)`), introducing a time-locked admin transfer mechanism that mitigates key compromise risk.
- Granular Role Hierarchy:
 - `ROLE_MANAGER_ROLE` — serves as the admin for all operational roles
 - `UG_ADMIN_ROLE` / `VAULT_ADMIN_ROLE` / `VPC_ADMIN_ROLE` — scoped to contract-specific configuration functions
 - `OPERATOR_ROLE` — restricted to operational updates (dependency addresses, unpause)
 - `PAUSER_ROLE` — limited to pause functionality
 - `TSS_ROLE` / `VAULT_ROLE` — retained for cross-contract trust boundaries
- Administrative functions previously gated by `DEFAULT_ADMIN_ROLE` are now assigned to purpose-specific roles:
 - Configuration setters (`setCapsUSD`, `setProtocolFee`, `setTokenLimitThresholds`, etc.) → contract-specific admin roles
 - Dependency updates (`updateTSS`, `updateVault`, `updateCEAFactory`, etc.) → `OPERATOR_ROLE`
 - Emergency pause → `PAUSER_ROLE`; unpause → `OPERATOR_ROLE`

The separation of concerns reduces the blast radius of a single key compromise and aligns with secure role-based access control practices for multi-admin protocol governance.

F-2026-15643 - updateEpochDuration Implicitly Resets Rate-Limit Usage for All Tokens - Medium

Description:

The function `updateEpochDuration` changes `epochDurationSec` immediately, but token usage accounting (`_usage[token].epoch / _usage[token].used`) is keyed by the derived epoch index `block.timestamp / epochDurationSec`.

```
function updateEpochDuration(uint256 newDurationSec) external onlyRole(DEFAULT_ADMIN_ROLE) whenNotPaused {
    if (newDurationSec == 0) revert Errors.InvalidInput();
    uint256 old = epochDurationSec;
    epochDurationSec = newDurationSec;
    emit EpochDurationUpdated(old, newDurationSec);
}
```

When `epochDurationSec` is modified, the computed epoch index changes globally. As a result, existing per-token usage entries no longer match the new epoch index and are treated as stale, causing implicit reset of used to zero on next consumption.

```
uint64 current = uint64(block.timestamp / _epochDuration);
EpochUsage storage e = _usage[token];
if (e.epoch != current) {
    e.epoch = current;
    e.used = 0;
}
```

A governance/admin action can instantly restore full throughput for all tokens without waiting for natural epoch rollover.

This can bypass intended throttling cadence and materially weaken the protocol's risk controls.

Even benign configuration updates may unintentionally wipe active usage state, causing unexpected behavior.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Mitigated

Classification

Impact Rate:	2/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Use one of these safer patterns:

- Two-step delayed activation: store `pendingEpochDurationSec` and activate at a known future boundary.
- Epoch-anchor model: maintain a fixed `epochStartTimestamp`; when duration changes, preserve continuity by remapping current usage to the new epoch.
- Explicit reset function: separate "change duration" from "reset usage," and require a dedicated privileged call/event for reset so behavior is intentional and auditable.

Resolution: **Mitigated in** `e85d683` :

The implicit rate-limit reset behavior remains, but it is now explicitly documented in `updateEpochDuration` NatSpec. `EpochDurationUpdated` was extended with `epochIndexAtChange`, allowing off-chain monitors to detect and audit the reset effect caused by changing `epochDurationSec`. No on-chain constraint, timelock, delayed activation, or preservation of existing usage state was added, so the risk is reduced through transparency rather than fully eliminated.

[F-2026-15646](#) - Documentation and Implementation Mismatch in Token Support Validation - Medium

Description:

The documentation specifies that revert processing in the `Vault` contract validates token allowlist support before executing recovery flows. However, the implementation of `Vault.revertUniversalTx()` and `Vault.rescueFunds()` does not enforce any token support validation. According to the documentation:

Revert_Handling.md

Lines 46-48

1. TSS calls ``Vault.revertUniversalTx(subTxId, universalTxId, token, amount, revertInstruction)``.
2. Vault validates: ``amount > 0`, `revertRecipient != address(0)`, token is supported, `IERC20(token).balanceOf(vault) >= amount``.
3. Vault calls ``IERC20(token).safeTransfer(gateway, amount)``.

4_OutboundTx_Flows.md

Lines 887-889

```
TSS → Vault.revertUniversalTxToken(subTxId, uSubTxId, USDC, amount, revertInstruction)
    → Vault: validate token support + balance, safeTransfer(gateway, amount)
    → gateway.revertUniversalTxToken(...) → safeTransfer(revertRecipient, amount)
```

Instead, the implementation only verifies basic invariants such as non-zero amount, valid recipient address, and sufficient token balance before forwarding the transfer to the gateway. No check is performed to ensure that the token is part of a supported or allowlisted set.

```
function revertUniversalTx(bytes32 subTxId, bytes32 universalTxId, address token,
uint256 amount,
RevertInstructions calldata revertInstruction) external payable
nonReentrant whenNotPaused onlyRole(TSS_ROLE) {
    _validateRevertParams(amount, revertInstruction.revertRecipient);

    if (token == address(0)) {
        if (msg.value != amount) revert Errors.InvalidAmount();
        gateway.
            revertUniversalTx{value : amount}(subTxId, universalTxId, token, amount, revertInstruction);
    }
```

```

    } else {
        if (msg.value != 0) revert Errors.InvalidAmount();
        if (IERC20(token).balanceOf(address(this)) < amount) {
            revert Errors.InsufficientBalance();
        }
        IERC20(token).safeTransfer(address(gateway), amount);
        gateway.revertUniversalTx(subTxId, universalTxId, token, amount, revertInstruction);
    }

    emit UniversalTxReverted(subTxId, universalTxId, token, amount, revertInstruction);
}

function rescueFunds(bytes32 subTxId, bytes32 universalTxId, address token, uint256 amount,
                    RevertInstructions calldata revertInstruction) external payable nonReentrant
    whenNotPaused onlyRole(TSS_ROLE) {
    _validateRevertParams(amount, revertInstruction.revertRecipient);

    if (token == address(0)) {
        if (msg.value != amount) revert Errors.InvalidAmount();
        gateway.rescueFunds{value : amount}(subTxId, universalTxId, token, amount, revertInstruction);
    } else {
        if (msg.value != 0) revert Errors.InvalidAmount();
        if (IERC20(token).balanceOf(address(this)) < amount) {
            revert Errors.InsufficientBalance();
        }
        IERC20(token).safeTransfer(address(gateway), amount);
        gateway.rescueFunds(subTxId, universalTxId, token, amount, revertInstruction);
    }

    emit FundsRescued(subTxId, universalTxId, token, amount, revertInstruction);
}

function _validateRevertParams(uint256 amount, address revertRecipient) private pure {
    if (amount == 0) revert Errors.InvalidAmount();
    if (revertRecipient == address(0)) revert Errors.InvalidRecipient();
}

```

In contrast, the actual implementation omits the token support validation step. This creates a discrepancy between documented behavior and executed logic. As a result, revert operations may be performed for tokens

that are not intended to be supported, leading to incorrect assumptions regarding recovery policies and asset handling constraints.

Assets:

- Vault.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:Fixed

Classification**Impact Rate:** 2/5**Likelihood Rate:** 5/5**Exploitability:** Semi-Dependent**Complexity:** Simple**Severity:** Medium

Recommendations**Remediation:**

It is recommended to align the implementation with the documented behavior by introducing explicit token allowlist validation within `revertUniversalTx()` and `rescueFunds()`, or alternatively updating the documentation to accurately reflect the absence of such validation. Ensuring consistency between documentation and implementation reduces ambiguity and prevents incorrect assumptions about asset recovery constraints.

Resolution:**Fixed in** a81510e :

The remaining documentation mismatch was corrected. 4_OutboundTx_Flows.md no longer states that Vault validates token support during ERC20 revert handling and now describes the implemented validation as amount and balance checks. The implementation remains unchanged, as the issue was documentation-level.

[F-2026-15656](#) - `_burnPRC20` Ignores `transferFrom` Return Value, Allowing Silent Transfer Failure - Medium

Description: `_burnPRC20` function in **UniversalGatewayPC** contract calls `transferFrom` on the PRC20 token, but does not verify its return value:

```
function _burnPRC20(address from, address token, uint256 amount) internal {
    IPRC20(token).transferFrom(from, address(this), amount);
    bool ok = IPRC20(token).burn(amount);
    if (!ok) revert Errors.TokenBurnFailed(token, amount);
}
```

The corresponding interface explicitly defines `transferFrom` as returning `bool`:

```
interface IPRC20 {
    function burn(uint256 amount) external returns (bool);
    function transferFrom(address sender, address recipient, uint256 amount) external returns (bool);
}
```

This means the token may signal failure by returning false rather than reverting. Since the return value is ignored, `_burnPRC20` may continue execution under the false assumption that the tokens were successfully transferred into the gateway.

`sendUniversalTxOutbound` accepts an arbitrary `req.token` address from the caller. The only validation is a non-zero check:

```
function _validateParams(address token, address revertRecipient) internal pure {
    if (token == address(0)) revert Errors.ZeroAddress();
    if (revertRecipient == address(0)) {
        revert Errors.InvalidRecipient();
    }
}
```

1. `getOutboundTxGasAndFees(req.token, ...)` — calls `IPRC20(req.token).SOURCE_CHAIN_NAMESPACE` to resolve chain. Any contract implementing this function will pass.
2. `_burnPRC20(msg.sender, req.token, req.amount)` — calls `transferFrom` and `burn` on the user-supplied address:

There is no verification that `req.token` is a legitimate **PRC20** registered via `isSupportedToken` in **UniversalCore**.

```
mapping(address => bool) public isSupportedToken;
```

This mapping exists in **UniversalCore** but is **never checked** during the outbound flow. The token address passes through two external calls without allowlist validation:

If a **PRC20** implementation returns false on failed transfer instead of reverting:

- the gateway may proceed as if the user's tokens were collected;
- subsequent logic may still attempt to burn tokens or emit outbound events;
- the system risks accounting inconsistencies between burned Push-side representation and source-chain release logic;
- in the worst case, this can enable value extraction from the source-side vault without proper asset backing on Push Chain.

This is particularly dangerous because `_burnPRC20` is part of the core outbound flow in `sendUniversalTxOutbound`.

Attack Scenario

1. Attacker deploys a malicious contract at address **FakeToken** that implements:

- `SOURCE_CHAIN_NAMESPACE` → returns a valid chain namespace string (e.g., "eip155:1")
- `transferFrom` → returns true without moving any tokens
- `burn` → returns true without destroying anything

1. Attacker calls `sendUniversalTxOutbound` with:

- `req.token` = `FakeToken`
- `req.amount` = `1_000_000e18` (arbitrary large amount)
- Sufficient `msg.value` to cover gas fee + protocol fee

1. Execution flow:

- `_validateParams`: passes (`token != address(0)`)
- `_fetchOutboundTxGasAndFees`: calls `FakeToken.SOURCE_CHAIN_NAMESPACE` → returns valid namespace → gas fee is resolved normally
- `_burnPRC20`: calls `FakeToken.transferFrom` → returns true (no real transfer). Return value ignored. Calls `FakeToken.burn` → returns true (no real burn)
- `_swapAndCollectFees`: processes normally (gas fee swap uses legitimate gasToken)
- `UniversalTxOutbound` event is emitted with `token = FakeToken`, `amount = 1_000_000e18`

1. TSS or off-chain relayer observes the event and, depending on validation logic:
 - If it does not verify token legitimacy: unlocks real tokens on the source chain
 - If it does verify: the attack fails at the off-chain layer

UniversalGateway uses **SafeERC20** for **ERC20** transfers, while **UniversalGatewayPC** does not apply the same defensive pattern for **PRC20** handling. That inconsistency increases integration risk and makes this path more fragile.

Assets:

- UniversalGatewayPC.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Validate the return value explicitly, or use a safe token wrapper that enforces success semantics.

Resolution: Fixed in `80f7a20` :
`_burnPRC20` now captures the `transferFrom` return value and reverts with `Errors.TokenTransferFailed(token, amount)` when the transfer reports failure. False-success PRC20 transfer semantics can no longer be ignored. Regression coverage was added for unsupported-token rejection and for a mock PRC20 whose `transferFrom` returns `false`.

Evidences

PoC: Attacker Uses FakeToken to Emit Outbound Event

Reproduce:

```
function test_FakeToken_OutboundEvent_WithoutRealBurn() public {
    uint256 fakeAmount = 1_000_000e18; // claim to bridge 1M tokens

    UniversalOutboundTxRequest memory req = UniversalOutboundTxRequest({
        recipient: bytes(""),
        token: address(fakeToken),          // malicious token
        amount: fakeAmount,
        gasLimit: DEFAULT_GAS_LIMIT,
        payload: bytes(""),                 // FUNDS type
        revertRecipient: attacker
    });

    // Record logs to inspect emitted events
    vm.recordLogs();

    vm.prank(attacker);
    gateway.sendUniversalTxOutbound{ value: PC_FEE }(req);

    // ---- Verify the event was emitted with the fake amount ----
    Vm.Log[] memory logs = vm.getRecordedLogs();

    // Find UniversalTxOutbound event
    bytes32 eventSig = keccak256(
        "UniversalTxOutbound(bytes32,address,string,address,bytes,uint256
, address,uint256,uint256,bytes,uint256,address,uint8,uint256)"
    );

    bool eventFound = false;
    for (uint256 i = 0; i < logs.length; i++) {
        if (logs[i].topics.length > 0 && logs[i].topics[0] == eventSig) {
            eventFound = true;
            // The event's indexed token (topics[3]) should be the fake token

            assertEq(
                logs[i].topics[3],
                bytes32(uint256(uint160(address(fakeToken)))),
                "Event token should be FakeToken"
            );
            break;
        }
    }
}
```



```

    assertTrue(eventFound, "UniversalTxOutbound event must be emitted");

    // ---- Confirm NO real tokens were moved ----
    // FakeToken has no internal balances – nothing was transferred or burned.
    // The nonce incremented, meaning the gateway treated this as a valid outbound tx.
    assertEquals(gateway.nonce(), 1, "Nonce should have incremented - tx was accepted");
}

/// @notice Secondary check: the same attack works for FUNDS_AND_PAYLOAD type
function test_FakeToken_FundsAndPayload_WithoutRealBurn() public {
    uint256 fakeAmount = 5_000_000e18;
    bytes memory payload = abi.encodeWithSignature("execute()");

    UniversalOutboundTxRequest memory req = UniversalOutboundTxRequest({
        recipient: bytes(""),
        token: address(fakeToken),
        amount: fakeAmount,
        gasLimit: DEFAULT_GAS_LIMIT,
        payload: payload, // FUNDS_AND_PAYLOAD type
        revertRecipient: attacker
    });

    uint256 nonceBefore = gateway.nonce();

    vm.prank(attacker);
    gateway.sendUniversalTxOutbound{ value: PC_FEE }(req);

    // Tx succeeded – event emitted with 5M fake tokens
    assertEquals(gateway.nonce(), nonceBefore + 1, "Nonce should increment for FUNDS_AND_PAYLOAD too");
}

```

Results:

```

[PASS] test_FakeToken_FundsAndPayload_WithoutRealBurn() (gas: 155170)
[PASS] test_FakeToken_OutboundEvent_WithoutRealBurn() (gas: 163047)

```

[F-2026-15658](#) - rescueFundsOnSourceChain Lacks Replay Protection and universalTxId-Token Binding - Medium

Description:

`rescueFundsOnSourceChain` allows any caller to emit a rescue request event for any (`universalTxId`, `prc20`) pair, but the function does not:

1. enforce one-time execution per `universalTxId` (no deduplication / replay protection), and
2. verify that the provided `prc20` actually corresponds to the provided `universalTxId` (no parameter binding).

```
function rescueFundsOnSourceChain(
    bytes32 universalTxId,
    address prc20
) external payable whenNotPaused nonReentrant {
    if (prc20 == address(0)) revert Errors.ZeroAddress();
    (
        address gasToken,
        uint256 gasFee,
        uint256 rescueGasLimit,
        uint256 gasPrice,
        string memory chainNamespace
    ) = IUniversalCore(UNIVERSAL_CORE).getRescueFundsGasLimit(prc20);
    _swapAndCollectFees(gasToken, msg.value, gasFee);
    emit RescueFundsOnSourceChain(
        universalTxId,
        prc20,
        chainNamespace,
        msg.sender,
        TX_TYPE.RESCUE_FUNDS,
        gasFee,
        gasPrice,
        rescueGasLimit
    );
}
```

Unlike **UniversalGateway** revert/rescue handling (which uses `isExecuted[subTxId]`), this path has no equivalent execution guard, leading to:

- **Replay/Event spam risk:** the same `universalTxId` can be submitted repeatedly, producing multiple rescue events.
- **Semantic mismatch risk:** callers can pair a real `universalTxId` with an unrelated `prc20`, producing ambiguous or misleading rescue signals.

- **Operational risk for relayers/TSS:** if off-chain processors are not strictly idempotent and strongly validating, duplicate or mismatched rescue attempts may trigger repeated processing attempts, wrong-route execution attempts, or manual incident overhead.

Even if no direct fund loss occurs on-chain, this is a protocol-integrity and off-chain safety issue because rescue execution is event-driven.

Attack / Failure Scenario

1. A valid `universalTxId` becomes publicly known from prior outbound flow.
2. An attacker repeatedly calls `rescueFundsOnSourceChain(universalTxId, arbitraryPrc20)` and pays gas.
3. Multiple `RescueFundsOnSourceChain` events are emitted for the same ID, potentially with different token contexts.
4. Off-chain processors must deduplicate and validate; otherwise, duplicate/wrong rescue handling may be attempted.

Assets:

- UniversalGatewayPC.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Accepted

Classification

Impact Rate:	4/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation:

- Add replay protection:
 - `mapping(bytes32 => bool) public rescueRequested;`
 - revert if `rescueRequested[universalTxId]` is already `true`.
- Add binding validation:
 - either persist canonical outbound metadata on-chain and validate (`universalTxId`, `prc20`) before emit,
 - or require a signed proof from trusted authority tying `universalTxId` to `prc20`.

- Emit an explicit status event when a rescue request is accepted/rejected to improve indexer behavior.

Resolution:

Accepted:

`rescueFundsOnSourceChain` still permits repeated submissions for the same `universalTxId` and still does not bind `prc20` to that identifier on Push Chain. The client identified this behavior as intentional because the function only emits an event and does not release assets. Replay protection for actual fund release remains enforced on the source chain through `UniversalGateway._validateRevertParams` and `isExecuted[subTxId]`, while repeated emission is retained to preserve retryability when a rescue event is not picked up by relayers. Event-level duplication and token-context validation therefore remain delegated to off-chain processing.

[F-2026-15731](#) - Inconsistent Recipient Semantics Between Documentation and Funds Event Emission - Medium

Description:

The protocol documentation defines a clear semantic rule for inbound event routing. In non-CEA flows, `recipient` is expected to be `address(0)`, while non-zero values are reserved exclusively for CEA-originated routing. This distinction is significant, as relayers and indexers may rely on the `recipient` field as a routing signal when determining whether to derive the destination UEA from `sender` or to use an explicitly provided address.

The documented behavior specifies:

```
5_InboundTx_Flows.md
```

```
Lines 46-48
```

```
- **`recipient == address(0)`** in `UniversalTx`: Push Chain attributes the i  
nbound tx to  
  `sender`'s own UEA.  
- **`recipient != address(0)`** (CEA path only): Push Chain routes to that ex  
plicit UEA address.
```

```
CLAUDE.md
```

```
Lines 474-477
```

```
- `req.recipient` must always equal the CEA's mapped UEA – never `address(0)`  
, never an arbitrary address  
- All events emitted via CEA path have `fromCEA=true` and `recipient=mappedUE  
A`  
- Normal `sendUniversalTx` path is unchanged: still emits `fromCEA=false` and  
`recipient=address(0)`
```

In the implementation, this rule is partially enforced. Within `_sendTxWithFunds`, the gas-related event correctly applies normalization:

```
if (nativeValue > 0) {  
  address gasRecipient = fromCEA ? _req.recipient : address(0);  
  _sendTxWithGas(  
    TX_TYPE.GAS,  
    _msgSender(),  
    gasRecipient, n  
    ativeValue,  
    bytes(""),  
    _req.revertRecipient,  
    _req.signatureData,  
    fromCEA
```

```
);
}
```

However, in the `TX_TYPE.FUNDS` branch, the emitted event uses `_req.recipient` directly without enforcing normalization for non-CEA flows:

```
_emitUniversalTx(
  _msgSender(),
  _req.recipient,
  tokenForFunds,
  _req.amount,
  _req.payload,
  _req.revertRecipient,
  txType,
  _req.signatureData,
  fromCEA
);
```

This results in inconsistent behavior, where non-CEA transactions may emit a non-zero `recipient`, contrary to documented expectations.

The `UniversalTx` event serves as a primary input for TSS (Off-chain Threshold Signature Scheme relay operated by Push Chain validators), responsible for cross-chain execution, including minting and routing logic. Inconsistent recipient semantics may lead to incorrect interpretation of routing intent, resulting in inconsistent accounting, incorrect asset attribution, or unintended crediting behavior in downstream systems.

5_InboundTx_Flows.md

Lines 7-9

```
`UniversalGateway` on the external chain is the sole entry point. It infers the transaction type,
enforces rate limits, moves funds, and emits the `UniversalTx` event that TSS
observes to mint
PRC20 or execute UEA payloads on Push Chain.
```

5_InboundTx_Flows.md

Lines 38-45

Inbound transactions result in PRC20 minting on Push Chain:

...

TSS observes the `UniversalTx` event and mints PRC20-ETH ...

...

TSS observes the `UniversalTx` event and mints the corresponding PRC20 token

...

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Accepted

Classification**Impact Rate:** 2/5**Likelihood Rate:** 5/5**Exploitability:** Independent**Complexity:** Simple**Severity:** Medium**Recommendations**

Remediation: It is recommended to enforce consistent recipient semantics by normalizing `recipient` to `address(0)` for all non-CEA `TX_TYPE.FUNDS` emissions.

Resolution: **Accepted:**

`_sendTxWithFunds` still emits `_req.recipient` unchanged on the `TX_TYPE.FUNDS` path, while only the gas leg and the `FUNDS_AND_PAYLOAD` path normalize `recipient` to `address(0)` for non-CEA routing. The current documentation still describes `recipient != address(0)` as CEA-only semantics, so the documented rule remains misaligned with the implementation. The client classified the asymmetric FUNDS-route behavior as intentional and chose documentation correction rather than code modification.

F-2026-15639 - Absence of Granular Role Separation - Low

Description:

The `UniversalGateway`, `UniversalGatewayPC` and `Vault` contract implement the `DEFAULT_ADMIN_ROLE`, which currently grants extensive privileges across multiple administrative operations, including setter functions. The wide scope of authority assigned to this role introduces a security concern when it is used for day-to-day administrative tasks.

A compromise of the private key controlling this role could result in full unauthorized access to key protocol functions. This may lead to unauthorized configuration changes, disruption of system behavior, or full control over protocol parameters. The current design lacks role segregation, where distinct operational and administrative responsibilities are assigned to separate roles with limited scopes of authority.

Assets:

- `UniversalGateway.sol` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]
- `Vault.sol` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]
- `UniversalGatewayPC.sol` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation:

It is recommended to introduce granular role segregation by creating separate role like `ADMIN_ROLE` for routine administrative operations and higher-privilege management functions. The `DEFAULT_ADMIN_ROLE` should be reserved exclusively for roles management.

Resolution:

Fixed in `43e8cc2` :

Granular role separation was introduced across **UniversalGateway**, **UniversalGatewayPC**, **Vault**, and **VaultPC** using **AccessControlDefaultAdminRulesUpgradeable**.

[F-2026-15655](#) - Missing Rescue Gas Limit Controls in UniversalGatewayPC Implementation - Low

Description:

The documentation specifies that `UniversalGatewayPC` includes a configurable `RESCUE_FUNDS_GAS_LIMIT` storage variable and an associated `setRescueFundsGasLimit(uint256)` administrative function. These components are described as part of the contract's gas management mechanism for rescue operations.

```
3_UniversalGatewayPC.md
```

```
Lines 13-14
```

```
| **UGPC_1: Admin Actions**      | `initialize`, `pause`/`unpause`, `setVaultP  
C`, `setRescueFundsGasLimit` |  
| **UGPC_2: Outbound TX**      | `sendUniversalTxOutbound`, `rescueFundsOnSo  
urceChain`                    |
```

```
3_UniversalGatewayPC.md
```

```
Lines 138-141
```

```
- Fixed gas limit via `RESCUE_FUNDS_GAS_LIMIT` (admin-configurable).  
...  
**Storage variable:** `RESCUE_FUNDS_GAS_LIMIT` – set via `setRescueFundsGasLi  
mit(uint256)` (admin only, whenNotPaused).
```

```
4_OutboundTx_Flows.md
```

```
Lines 1092-1095
```

```
1. Validate `prc20 != address(0)` and `RESCUE_FUNDS_GAS_LIMIT != 0`.  
...  
4. Compute `gasFee = gasPrice * RESCUE_FUNDS_GAS_LIMIT`.
```

However, the current implementation does not define the `RESCUE_FUNDS_GAS_LIMIT` storage variable, nor implement the `setRescueFundsGasLimit(uint256)` function. Instead, rescue gas parameters are retrieved dynamically from `UniversalCore` via `getRescueFundsGasLimit(prc20)`, making `UniversalCore` the effective source of truth for this configuration.

Assets:

- `UniversalGatewayPC.sol` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:	It is recommended to align the documentation and implementation by either introducing the <code>RESCUE_FUNDS_GAS_LIMIT</code> storage variable and corresponding setter function in <code>UniversalGatewayPC</code>, or updating the documentation to reflect that rescue gas parameters are sourced exclusively from <code>UniversalCore</code>. Consistency between documentation and implementation should be ensured to prevent incorrect assumptions regarding configuration and gas fee calculation.
Resolution:	Fixed in <code>01f466a</code> : The main UGPC and outbound-flow documentation was updated to remove references to local <code>RESCUE_FUNDS_GAS_LIMIT</code> storage and <code>setRescueFundsGasLimit</code>. The docs now state that rescue gas parameters are sourced from <code>UniversalCore.getRescueFundsGasLimit(prc20)</code> and that UGPC has no local rescue gas storage variable or setter.

[F-2026-15657](#) - UniversalGatewayPC Lacks an Admin Setter for UNIVERSAL_CORE - Low

Description: **UniversalGatewayPC** stores the `UNIVERSAL_CORE` dependency in state and initializes it once during initialize, but provides no admin function to update it later.

```
address public UNIVERSAL_CORE;
/// @custom:oz-upgrades-unsafe-allow constructor
constructor() {
    _disableInitializers();
}
IVaultPC public VAULT_PC;
function initialize(address admin, address pauser, address universalCore, address vaultPC) external initializer {
    if (admin == address(0) || pauser == address(0) || universalCore == address(0) || vaultPC == address(0)) {
        revert Errors.ZeroAddress();
    }
    __AccessControl_init();
    __ReentrancyGuard_init();
    __Pausable_init();
    _grantRole(DEFAULT_ADMIN_ROLE, admin);
    _grantRole(PAUSER_ROLE, pauser);
    UNIVERSAL_CORE = universalCore;
    VAULT_PC = IVaultPC(vaultPC);
}
```

Unlike `VAULT_PC`, which can be updated through `setVaultPC`, `UNIVERSAL_CORE` is immutable at the application level:

```
function setVaultPC(address vaultPC) external onlyRole(DEFAULT_ADMIN_ROLE) whenNotPaused {
    if (vaultPC == address(0)) revert Errors.ZeroAddress();
    address oldVaultPC = address(VAULT_PC);
    VAULT_PC = IVaultPC(vaultPC);
    emit VaultPCUpdated(oldVaultPC, vaultPC);
}
```

At the same time, `UNIVERSAL_CORE` is a critical dependency used for:

- quoting gas fees,
- resolving gas token / chain metadata,
- and executing `swapAndBurnGas`.

```
(
    address gasToken,
    uint256 gasFee,
    uint256 rescueGasLimit,
    uint256 gasPrice,
    string memory chainNamespace
) = IUniversalCore(UNIVERSAL_CORE).getRescueFundsGasLimit(prc20);
_swapAndCollectFees(gasToken, msg.value, gasFee);
gasLimitUsed = gasLimit == 0 ? IUniversalCore(UNIVERSAL_CORE).BASE_GAS_LIMIT(
) : gasLimit;
(gasToken, gasFee, protocolFee, gasPrice, chainNamespace) =
    IUniversalCore(UNIVERSAL_CORE).getOutboundTxGasAndFees(token, gasLimitUsed
);
// ...
IUniversalCore(UNIVERSAL_CORE).swapAndBurnGas{ value: pcAmount }(gasToken, 0,
gasFee, 0, msg.sender);
```

If **UniversalCore** must be replaced due to:

- a bug,
- an upgrade,
- router migration,
- pricing logic correction,
- or compromised configuration,

then **UniversalGatewayPC** cannot be re-pointed via normal admin operations. The only recovery path is a proxy upgrade or full redeployment/migration of the gateway layer. This increases operational risk and reduces the system's ability to respond quickly to failures in a core dependency.

Assets:

- UniversalGatewayPC.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

Add an admin-only setter for `UNIVERSAL_CORE`, similar to `setVaultPC`, for example:

```
function setUniversalCore(address newUniversalCore) external onlyRole(DEFAULT_ADMIN_ROLE) whenNotPaused {
    if (newUniversalCore == address(0)) revert Errors.ZeroAddress();
    address oldUniversalCore = UNIVERSAL_CORE;
    UNIVERSAL_CORE = newUniversalCore;
    emit UniversalCoreUpdated(oldUniversalCore, newUniversalCore);
}
```

Resolution:

Fixed in `911d6c6` :

UniversalGatewayPC now exposes `setUniversalCore(address)` for `DEFAULT_ADMIN_ROLE`, gated by `whenNotPaused`, with zero-address validation. The update emits `UniversalCoreUpdated(oldUniversalCore, newUniversalCore)`, allowing the gateway to be re-pointed to a replacement **UniversalCore** without a proxy upgrade or redeployment. Regression tests cover success, non-admin rejection, zero-address rejection, and paused-state rejection.

[F-2026-15659](#) - Users Have No Control Over Swap Parameters in Outbound Gas-Fee Conversion - Low

Description:

In the outbound flow, users cannot control key swap-risk parameters (fee tier, deadline, max input) when native PC is swapped into gas token for cross-chain execution fees. **UniversalGatewayPC** always calls `swapAndBurnGas` with hardcoded values:

- `fee = 0` (meaning "use default fee tier from **UniversalCore**")
- `deadline = 0` (meaning "use default deadline from **UniversalCore**")

```
function _swapAndCollectFees(address gasToken, uint256 pcAmount, uint256 gasFee) internal {
    if (pcAmount == 0) revert Errors.ZeroAmount();
    IUniversalCore(UNIVERSAL_CORE).swapAndBurnGas{ value: pcAmount }(gasToken, 0, gasFee, 0, msg.sender);
}
```

UniversalCore then resolves those defaults internally:

```
if (fee == 0) {
    fee = defaultFeeTier[gasToken];
    if (fee == 0) revert UniversalCoreErrors.InvalidFeeTier();
}
if (deadline == 0) {
    deadline = block.timestamp + (defaultDeadlineMins * 1 minutes);
}
```

The swap uses:

- exact output (`amountOut = gasFee`)
- `amountInMaximum = msg.value` (entire user-provided PC amount)

```
ISwapRouter.ExactOutputSingleParams memory params = ISwapRouter.ExactOutputSingleParams({
    tokenIn: WPC,
    tokenOut: gasToken,
    fee: fee,
    recipient: address(this),
    deadline: deadline,
    amountOut: gasFee,
    amountInMaximum: msg.value,
```

```
sqrtPriceLimitX96: 0  
});
```

Because users cannot set swap controls per transaction:

- No user-level slippage bound on input spend (max spend is always full `msg.value`).
- No per-tx deadline preference (relies on the global admin-configured default).
- No per-tx fee-tier choice (relies on global default tier, even if liquidity is poor there).

This exposes users to avoidable MEV and pricing risk:

- pool manipulation or volatility can consume more PC than expected,
- users cannot cap spend with a personalized `maxPCForGas`,
- users must trust global admin defaults for execution quality.

Assets:

- UniversalGatewayPC.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status: Mitigated

Classification

Impact Rate: 2/5
Likelihood Rate: 2/5
Exploitability: Independent
Complexity: Simple
Severity: Low

Recommendations

Remediation: Add user-configurable swap constraints in outbound requests, e.g.:

- `maxPCForGas` (hard cap on input spend),
- optional `swapFeeTier`,
- optional `swapDeadline`.

Then pass them into `swapAndBurnGas` instead of fixed (`fee=0`, `deadline=0`) and enforce:

- `amountInUsed <= maxPCForGas`,
- deadline consistency with user intent.

This preserves protocol flexibility while reducing user exposure to price manipulation and poor default routing.

Resolution:

Mitigated in [4249311](#) :

`UniversalOutboundTxRequest` now includes `maxPCForGas`, allowing users to cap the native PC amount forwarded into `UniversalCore.swapAndBurnGas`. If `maxPCForGas` is set, any excess PC above that cap is refunded before the swap, and the exact-output swap can no longer consume more than the user-specified amount.

The issue is not fully fixed because `fee` tier and `deadline` remain hardcoded as `0`, so they still rely on `UniversalCore` defaults. The primary slippage/max-input risk is reduced through `maxPCForGas`, but per-transaction fee-tier and deadline controls were not added.

[F-2026-15660](#) - setVault Migration Can Orphan Previously Locked ERC20 Funds in the Old Vault - Low

Description:

`setVault` function from **UniversalGateway** contract immediately rotates `VAULT_ROLE` from the old vault to the new vault and updates `VAULT` storage pointer. New inbound ERC20 deposits are sent to the new vault, but previously locked ERC20 balances remain in the old vault.

```
function setVault(address newVault) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (newVault == address(0)) revert Errors.ZeroAddress();
    address old = VAULT;
    // transfer role
    if (hasRole(VAULT_ROLE, old)) _revokeRole(VAULT_ROLE, old);
    _grantRole(VAULT_ROLE, newVault);
    VAULT = newVault;
    emit VaultUpdated(old, newVault);
}
```

At the same time, **ERC20** deposits always target the current `VAULT` value:

```
// Handle ERC20 token deposit to Vault
if (tokenToLimitThreshold[token] == 0) revert Errors.NotSupported();
IERC20(token).safeTransferFrom(_msgSender(), VAULT, amount);
```

Revert/rescue execution in **UniversalGateway** is restricted to addresses with `VAULT_ROLE`:

```
function revertUniversalTx(...) external payable nonReentrant whenNotPaused onlyRole(VAULT_ROLE) {
    _validateRevertParams(subTxId, amount, token, revertInstruction.revertRecipient);
}
```

So after migration:

- old vault still holds historical **ERC20** balances;
- old vault no longer has `VAULT_ROLE`, thus cannot call gateway revert/rescue paths;
- new vault has role but may not hold the historical balances needed for those legacy tx flows.

Historical **ERC20** liquidity in the old vault can become operationally stranded from the standard revert/rescue pipeline. This creates:

- broken handling for legacy transactions tied to old-vault balances,

- reliance on manual emergency operations outside normal protocol flow,
- increased risk of failed refunds/rescues for users.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]
- Vault.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status: Mitigated

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Implement a safe migration strategy, for example:

- Two-phase migration
 - Phase 1: grant `VAULT_ROLE` to new vault, keep old vault role temporarily.
 - Phase 2: migrate balances and settle in-flight txs.
 - Phase 3: revoke old vault role.
- On-chain migration helper
 - old vault transfers supported **ERC20** balances to new vault before role revocation.
- Compatibility mode
 - allow multiple active vaults for revert/rescue until legacy balances are drained.

Also emit explicit migration lifecycle events to make vault transition auditable by operators/relayers.

Resolution: Mitigated in `bbe42b0` :

`Vault.migrateTokens` was added to support safe migration of ERC20 balances and native ETH to a new vault before `gateway.updateVault(newVault)` is called. The function requires the old Vault to be paused and checks that the Gateway is also paused, reducing the risk of deposits or operational flows

occurring during migration. It emits `TokensMigrated` with migrated token amounts and native value.

The issue is mitigated rather than fully eliminated because `updateVault` can still be called directly without enforcing that migration has occurred. Correct operation depends on following the documented sequence: pause Gateway and old Vault, call `migrateTokens(newVault, tokens)`, then call `updateVault(newVault)`.

[F-2026-15683](#) - msg.value Is Permanently Trapped in sendUniversalTx(UniversalTokenTxRequest) - Low

Description: The `sendUniversalTx` function in the **UniversalGateway** contract is `payable`, but it never uses `msg.value` as transaction input. Instead, it derives `nativeValue` only from `swapToNative` output:

```
function sendUniversalTx(UniversalTokenTxRequest calldata reqToken) external payable nonReentrant whenNotPaused {  
    if (!_isCallerCEA()) revert Errors.InvalidInput();  
    // ...  
    uint256 nativeValue =  
        swapToNative(reqToken.gasToken, reqToken.gasAmount, reqToken.amountOut  
MinETH, reqToken.deadline);
```

Then, routing uses that `nativeValue` (swap result), not `msg.value`:

```
TX_TYPE txType = _fetchTxType(req, nativeValue);  
_routeUniversalTx(req, _msgSender(), nativeValue, txType, false);
```

So any ETH sent in `msg.value` is not accounted into the transaction amounting logic and can remain stuck in the contract balance.

Users can accidentally lose ETH by calling token-overload with non-zero `msg.value`. Trapped ETH accumulates over time and cannot be recovered through normal protocol functions.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

Hard reject accidental ETH in this function:

```
if (msg.value != 0) revert Errors.InvalidInput();
```

Optionally add a controlled rescue mechanism for already trapped ETH (admin-only, timelocked/governed).

Resolution:

Fixed in [eecfa36](#):

The token-as-gas overload now rejects any non-zero `msg.value` with `Errors.InvalidInput` before performing the swap. This prevents native funds from being accidentally sent to and trapped in `UniversalGateway`, since the overload derives its effective native amount exclusively from `_swapToNative` output. Regression tests were updated to assert non-zero `msg.value` reverts and that valid token-as-gas calls leave no trapped native balance in the gateway.

[F-2026-15684](#) - Transaction Type Is Inferred Before Fee Deduction, Causing Post-Fee Route Mismatch - Low

Description:

The contract infers `TX_TYPE` using the raw pre-fee native amount, but later executes routing logic using the post-fee native amount after `INBOUND_FEE` has been deducted.

Type inference happens in the public entrypoints before routing:

```
function sendUniversalTx(UniversalTxRequest calldata req) external payable no
nReentrant whenNotPaused {
    if (!_isCallerCEA()) revert Errors.InvalidInput();
    uint256 nativeValue = msg.value;
    TX_TYPE txType = _fetchTxType(req, nativeValue);
    _routeUniversalTx(req, _msgSender(), nativeValue, txType, false);
}
```

Then `_routeUniversalTx` deducts the protocol fee and continues using the already-inferred `txType`:

```
if (!fromCEA) {
    uint256 feeCollected;
    (nativeValue, feeCollected) = _collectInboundFee(nativeValue);
    totalProtocolFeesCollected += feeCollected;
}
// Route 1: GAS or GAS_AND_PAYLOAD → Instant route
if (txType == TX_TYPE.GAS || txType == TX_TYPE.GAS_AND_PAYLOAD) {
```

As a result, the route type is decided from one value, while execution uses another.

This can create semantically inconsistent transactions, especially when `msg.value` is equal to or only slightly above `INBOUND_FEE`:

- a transaction may be classified as `GAS` pre-fee,
- but after fee deduction, the effective gas amount becomes zero,
- and the contract still emits a `UniversalTx` event for the gas route.

This leads to:

- zero-value gas events,
- confusing user behavior,
- inconsistent off-chain interpretation of what was actually funded.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Ensure that transaction type inference is performed using the same effective value that routing will use. For example, revalidate route consistency after fee deduction before dispatch.

Resolution: Fixed in `45ace21`:

TX type inference was moved into `_routeUniversalTx` after `_collectInboundFee` runs. Public entrypoints no longer compute and pass `txType` using the raw pre-fee native value.

Current behavior:

- Non-CEA native path: fee is deducted first, then `_fetchTxType(req, nativeValue)` uses the effective post-fee amount.
- Token-as-gas path: `_swapToNative` output is passed into `_routeUniversalTx`, then protocol fee is deducted before TX type inference.
- CEA path: fee deduction is intentionally skipped, so inference uses the full provided native value.

This resolves the reported mismatch. A fee-only transaction with no payload and no funds no longer emits a zero-value `GAS` event; after fee deduction, `_fetchTxType` sees no payload, no funds, and zero native value, then reverts with `InvalidInput`. Payload-only transactions with zero post-fee native value may still route as `GAS_AND_PAYLOAD`, which is consistent with the existing payload-only behavior.

F-2026-15686 - Native FUNDS Route Does Not Support Gas Top-Up Batching - Low

Description:

In the `TX_TYPE.FUNDS` flow, the native-token branch requires the post-fee native amount to match `req.amount` exactly:

```
if (txType == TX_TYPE.FUNDS) {
    address tokenForFunds;
    // Case 1.1: Token to bridge is Native Token -> address(0)
    // req.amount = desired bridge amount (fee is additive: msg.value = req.am
    ount + INBOUND_FEE).
    // nativeValue is post-fee (= msg.value - INBOUND_FEE = req.amount).
    if (_req.token == address(0)) {
        if (_req.amount != nativeValue) revert Errors.InvalidAmount();
        tokenForFunds = address(0);
    }
}
```

This means native `FUNDS` transactions cannot include any additional native amount for Push-side gas top-up. Any excess native value beyond `req.amount` causes the transaction to revert. However, gas batching is supported in closely related flows:

- ERC20 `FUNDS`: extra native value is routed as a gas leg
- Native `FUNDS_AND_PAYLOAD`: `nativeValue` - `req.amount` is split into a separate gas leg
- ERC20 `FUNDS_AND_PAYLOAD`: full native value can be used for gas

So the restriction exists only for the native `FUNDS` route without payload.

This creates an inconsistent user experience and unnecessary friction:

- users cannot bridge native funds and top up Push-side gas in a single no-payload transaction;
- they must either submit two separate transactions (`FUNDS + GAS`) or attach an otherwise unnecessary payload to access the split-flow behavior;
- this increases operational complexity and transaction cost.

While this is not a direct loss-of-funds vulnerability, it is a protocol behavior inconsistency that makes one valid user workflow artificially unavailable.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Support optional native gas batching in the native **FUNDS** route as well, for example by:

- allowing `nativeValue >= req.amount`, and
- routing `nativeValue - req.amount` as a separate **GAS** leg when non-zero.

This would align native **FUNDS** behavior with the other batching-enabled flows and eliminate the current inconsistency.

Resolution:

Accepted:

The native **TX_TYPE.FUNDS** route still requires the post-fee native value to equal `req.amount`, so native funds bridging cannot include an additional gas top-up in the same no-payload transaction. The client acknowledged the behavior as a UX inconsistency and retained the current invariant for v1 to avoid changing TX type inference late in the release. No assets are at risk, and alternative flows remain available through separate **FUNDS** and **GAS** transactions or the **FUNDS_AND_PAYLOAD** path.

[F-2026-15740](#) - Refund to Non-Receiving Contract Can Revert and Block Entire Outbound Transaction - Low

Description:

UniversalGatewayPC forwards outbound swap execution to `swapAndBurnGas` from **UniversalCore** and sets the refund recipient as `msg.sender`:

```
function _swapAndCollectFees(address gasToken, uint256 pcAmount, uint256 gasFee) internal {
    if (pcAmount == 0) revert Errors.ZeroAmount();
    IUniversalCore(UNIVERSAL_CORE).swapAndBurnGas{ value: pcAmount }(gasToken,
    0, gasFee, 0, msg.sender);
}
```

In **UniversalCore**, swap uses `exactOutputSingle` (targeting exact `gasFee` output), so unused native PC is typically refunded:

```
refund = msg.value - amountInUsed;
if (refund > 0) {
    IWPC(WPC).withdraw(refund);
    (bool ok,) = caller.call{value: refund}("");
    if (!ok) revert CommonErrors.TransferFailed();
}
```

If caller is a smart contract that does not accept native transfers (no payable receive/fallback, or explicitly reverts), refund transfer fails and **UniversalCore** reverts with `TransferFailed`. This reverts the entire outbound flow in `sendUniversalTxOutbound`.

Any contract account that cannot receive native refunds is unable to use outbound gateway flows that produce a refund (which is common with exact-output swaps).

Result: transaction-level DoS for those users/integrators. Affected integrations can include:

- multisigs without payable receive,
- routers/proxy wallets with strict ETH handling,
- contracts intentionally rejecting native transfers.

The design uses exact-output swaps (`amountOut = gasFee, amountInMaximum = msg.value`), which naturally creates refund scenarios unless the user sends exact quote precision. Therefore, this failure mode is not edge-only; it can occur in normal operation.

Assets:

- UniversalGatewayPC.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Accepted

Classification**Impact Rate:** 3/5**Likelihood Rate:** 2/5**Exploitability:** Independent**Complexity:** Simple**Severity:** Low**Recommendations****Remediation:**

Use one of these robust patterns:

- Credit-based refund model:
 - On failed native refund transfer, store `pendingRefund[caller] += refund`,
 - emit event,
 - allow caller to claim via explicit `withdrawRefund`.
- Configurable refund recipient:
 - Add `refundRecipient` to outbound request,
 - require it to be payable and user-controlled (EOA or known payable contract).

Resolution:**Accepted:**

`UniversalGatewayPC._swapAndCollectFees` still passes `msg.sender` as the refund recipient to `UniversalCore.swapAndBurnGas`, and failed native PC refunds still revert the outbound transaction. The client acknowledged the integration limitation and classified the behavior as intentional for this release. The remaining risk is limited to contract callers that cannot receive native PC refunds, with failure occurring deterministically as a full transaction revert rather than silent asset loss. Remediation was deferred in favor of documentation and caller-side guidance.

[F-2026-15766](#) - Misleading TSS Control Description in UniversalGateway Documentation - Low

Description: The top-level comment in `UniversalGateway` describes two "TSS-controlled functionalities." However, both statements are imprecise at the contract level and may lead to incorrect assumptions regarding authorization boundaries and token handling.

```
/**
 * @dev TSS-controlled functionalities:
 *
 * 1. TSS-controlled withdraw (native or ERC20).
 *
 * 2. Token Support List: allowlist for ERC20 used as gas inputs.
 */
```

First, the comment indicates that withdrawal and recovery operations are TSS-controlled. In the implementation and execution of the revert and rescue operations within `UniversalGateway` is gated by `VAULT_ROLE`, not `TSS_ROLE`. Enforcement of `TSS_ROLE` occurs upstream in the `Vault` contract, which invokes `UniversalGateway`. As a result, the immediate authorization boundary within `UniversalGateway` is based on trust in the `Vault`, rather than direct TSS control.

Second, the comment describes the token support list as an allowlist for ERC20 tokens used as gas inputs. In practice, the `tokenToLimitThreshold` mapping is managed by `DEFAULT_ADMIN_ROLE` and is used for rate-limiting and support validation of bridged ERC20 funds through mechanisms such as `_consumeRateLimit` and `_handleDeposits`. This mapping is not directly used to validate gas tokens in `sendUniversalTx(UniversalTokenTxRequest)`, where validation is limited to non-zero and swap-related constraints.

These discrepancies create a mismatch between documented behavior and actual implementation, potentially leading to incorrect assumptions about authorization flow and token validation logic.

Assets:

- `UniversalGateway.sol` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate:	4/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation: It is recommended to update the top-level contract comments to accurately reflect the actual authorization boundaries and token handling logic. The description of withdraw control should clarify the role of `VAULT_ROLE` within the contract, and the token support list description should be aligned with its actual use in rate-limiting and ERC20 fund handling rather than gas token validation.

Resolution: **Fixed in `045b911` :**

The misleading "TSS-controlled functionalities" NatSpec block has been replaced with an "Authorization model" section that accurately describes the contract behavior. The updated documentation clarifies that revert and rescue paths are gated by `VAULT_ROLE` with TSS authorization enforced upstream in the Vault contract, and that the `tokenToLimitThreshold` mapping is used for rate-limiting and bridge-support validation of ERC20 funds rather than gas token validation. Documentation-only change with no code logic affected.

[F-2026-15636](#) - Missing Event Emissions for Critical Admin Configuration Changes - Info

Description:

UniversalGateway contract allows the admin to modify multiple security-sensitive and routing-critical parameters through

`onlyRole(DEFAULT_ADMIN_ROLE)` functions, but several of these updates do not emit events. This creates an observability gap for off-chain monitoring, incident detection, and operational transparency.

```
function setTSS(address newTSS) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (newTSS == address(0)) revert Errors.ZeroAddress();
    address old = TSS_ADDRESS;
    // transfer role
    if (hasRole(TSS_ROLE, old)) _revokeRole(TSS_ROLE, old);
    _grantRole(TSS_ROLE, newTSS);
    TSS_ADDRESS = newTSS;
}

function setBlockUsdCap(uint256 cap1e18) external onlyRole(DEFAULT_ADMIN_ROLE)
    whenNotPaused {
    BLOCK_USD_CAP = cap1e18;
}

function setDefaultSwapDeadline(uint256 deadlineSec) external onlyRole(DEFAULT_ADMIN_ROLE) whenNotPaused {
    if (deadlineSec == 0) revert Errors.InvalidAmount();
    defaultSwapDeadlineSec = deadlineSec;
}

function setRouters(address factory, address router) external onlyRole(DEFAULT_ADMIN_ROLE) whenNotPaused {
    if (factory == address(0) || router == address(0)) revert Errors.ZeroAddress();
    uniV3Factory = IUniswapV3Factory(factory);
    uniV3Router = ISwapRouterV3(router);
}

function setV3FeeOrder(uint24 a, uint24 b, uint24 c) external onlyRole(DEFAULT_ADMIN_ROLE) whenNotPaused {
    v3FeeOrder = [a, b, c];
}

function setChainlinkStalePeriod(uint256 stalePeriodSec) external onlyRole(DEFAULT_ADMIN_ROLE) whenNotPaused {
    chainlinkStalePeriod = stalePeriodSec;
}
```

```

    }

    function setL2SequencerFeed(address feed) external onlyRole(DEFAULT_ADMIN_ROLE) whenNotPaused {
        l2SequencerFeed = AggregatorV3Interface(feed);
    }

    function setL2SequencerGracePeriod(uint256 gracePeriodSec) external onlyRole(DEFAULT_ADMIN_ROLE) whenNotPaused {
        l2SequencerGracePeriodSec = gracePeriodSec;
    }

    function setCEAFactory(address newFactory) external onlyRole(DEFAULT_ADMIN_ROLE) {
        if (newFactory == address(0)) revert Errors.ZeroAddress();
        CEA_FACTORY = newFactory;
    }
}

```

The absence of events does not directly break protocol execution, but it significantly weakens operational security and transparency:

- Off-chain monitoring systems cannot reliably detect critical configuration changes.
- Changes to TSS, router addresses, oracle settings, or **CEA** factory configuration may go unnoticed by indexers, watchdogs, and alerting infrastructure.
- Incident response and forensic analysis become more difficult because there is no event trail for sensitive administrative actions.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Add explicit events for every critical admin configuration update in **UniversalGateway**, for example:

- `TSSUpdated(address oldTss, address newTss)`
- `BlockUsdCapUpdated(uint256 oldCap, uint256 newCap)`
- `DefaultSwapDeadlineUpdated(uint256 oldDeadline, uint256 newDeadline)`
- `RoutersUpdated(address oldFactory, address newFactory, address oldRouter, address newRouter)`
- `V3FeeOrderUpdated(uint24 oldA, uint24 oldB, uint24 oldC, uint24 newA, uint24 newB, uint24 newC)`
- `ChainlinkStalePeriodUpdated(uint256 oldValue, uint256 newValue)`
- `L2SequencerFeedUpdated(address oldFeed, address newFeed)`
- `L2SequencerGracePeriodUpdated(uint256 oldValue, uint256 newValue)`
- `CEAFactoryUpdated(address oldFactory, address newFactory)`

Resolution:

The issue was accepted with the following statement:

The report lists 9 admin functions without event emissions. While we agree that event emissions improve observability, our approach is to **only add events for functions that are actually monitored by off-chain systems**.

F-2026-15638 - Redundant TSS_ROLE Management (Role Not Used for Authorization) - Info

Description:

In **UniversalGateway**, **TSS_ROLE** is granted and rotated, but it is not used anywhere for access control (`onlyRole(TSS_ROLE)` or equivalent checks).

Operationally, the contract relies on **TSS_ADDRESS** for value routing, not on role membership.

```
_grantRole(DEFAULT_ADMIN_ROLE, admin);
_grantRole(PAUSER_ROLE, pauser);
_grantRole(TSS_ROLE, tss);
_grantRole(VAULT_ROLE, vaultAddress);
TSS_ADDRESS = tss;

function setTSS(address newTSS) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (newTSS == address(0)) revert Errors.ZeroAddress();
    address old = TSS_ADDRESS;
    // transfer role
    if (hasRole(TSS_ROLE, old)) _revokeRole(TSS_ROLE, old);
    _grantRole(TSS_ROLE, newTSS);
    TSS_ADDRESS = newTSS;
}
```

At the same time, critical logic uses **TSS_ADDRESS** directly:

```
(bool ok,) = payable(TSS_ADDRESS).call{ value: fee }("");
if (!ok) revert Errors.DepositFailed();
```

This creates misleading security signaling (role appears meaningful but is not enforced) and adds unnecessary state management and gas overhead (grant/revoke operations with no functional effect).

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider removing `TSS_ROLE` entirely if `TSS_ADDRESS` is the only intended authority path.

Resolution: **Fixed in `bdaeabf` :**

`TSS_ROLE` was removed from **UniversalGateway**. The contract no longer declares `TSS_ROLE`, no longer grants it during initialization, and no longer rotates role membership when updating the TSS address.

UniversalGateway now only tracks `TSS_ADDRESS` for native deposit and protocol fee routing, while actual TSS authorization remains enforced in `Vault` through `onlyRole(TSS_ROLE)`. This removes the misleading unused role and eliminates the redundant grant/revoke state management in **UniversalGateway**.

F-2026-15640 - Admin Front-Run Risk on Protocol Fee Updates - Info

Description:

`setProtocolFee` lets `DEFAULT_ADMIN_ROLE` change `INBOUND_FEE` instantly, with no timelock, delay, or user-side fee bound.

```
function setProtocolFee(uint256 fee) external onlyRole(DEFAULT_ADMIN_ROLE) {
    INBOUND_FEE = fee;
    emit ProtocolFeeUpdated(fee);
}
```

The fee is enforced during execution (`_collectInboundFee`), not at signing/submission time:

```
function _collectInboundFee(uint256 nativeValue) private returns (uint256 adjustedNative, uint256 feeCollected) {
    uint256 fee = INBOUND_FEE;
    if (fee == 0) return (nativeValue, 0);
    if (nativeValue < fee) revert Errors.InsufficientProtocolFee();
}
```

This means an admin can raise fee right before user tx inclusion (same block mempool race), effectively front-running pending user transactions.

This may lead to:

- **Selective DoS / censorship:** pending txs revert with `InsufficientProtocolFee`.
- **User unpredictability:** for txs where `msg.value` still covers new fee, post-fee native amount changes (can alter gas leg behavior or make checks fail later).

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Mitigated

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity:

Info

Recommendations**Remediation:**

- Add timelocked fee updates (e.g., `proposeFee` + `executeFee` after delay).
- Add user-provided max acceptable fee parameter in request (`maxInboundFee`) and revert if `INBOUND_FEE > maxInboundFee`.
- Optionally enforce rate limits on fee changes (max delta per period) to reduce abrupt governance shocks.

Resolution:**Mitigated in `f8e9693` and `c1e28ac`:**

`setProtocolFee` now enforces `fee <= MAX_INBOUND_FEE`, and the maximum fee was reduced to `0.05 ether`. This limits the impact of sudden admin fee changes and prevents arbitrary fee spikes.

The issue is not fully fixed. `INBOUND_FEE` can still be changed immediately by `DEFAULT_ADMIN_ROLE`, and user requests do not include a per-transaction maximum accepted protocol fee. Pending transactions can still revert or execute with a changed post-fee native amount after an admin update, within the hard cap. No timelock, delayed activation, or user-side fee bound was added.

[F-2026-15641](#) - Unbounded setProtocolFee Allows Arbitrary Fee Configuration - Info

Description:

`setProtocolFee` allows `DEFAULT_ADMIN_ROLE` to set `INBOUND_FEE` to any `uint256` value, with no upper bound or sanity checks.

```
function setProtocolFee(uint256 fee) external onlyRole(DEFAULT_ADMIN_ROLE) {
    INBOUND_FEE = fee;
    emit ProtocolFeeUpdated(fee);
}
```

The configured fee is enforced in `_collectInboundFee` for non-CEA paths:

```
// Every tx must supply at least INBOUND_FEE in native
if (nativeValue < fee) revert Errors.InsufficientProtocolFee();
```

Because the fee is unbounded, admin can set it to an extremely high value that makes normal user flows economically or technically unusable.

This may lead to:

- **Protocol-level DoS vector (governance-controlled):** setting fee too high causes widespread reverts with `InsufficientProtocolFee`.
- **Economic griefing risk:** users may overpay to satisfy excessive fee settings.
- **Operational instability:** no guardrails to prevent accidental misconfiguration or abrupt fee spikes.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

- Enforce a hard cap, e.g. `MAX_INBOUND_FEE`, and validate `fee <= MAX_INBOUND_FEE`.
- Optionally enforce bounded step changes (max delta per update or per time window).
- Add governance safety controls (timelock / two-step update) to reduce misconfiguration and abuse risk.

Resolution:

Fixed in `f8e9693` :

`setProtocolFee` now enforces `fee <= MAX_INBOUND_FEE` and reverts with `Errors.InvalidInput` when the configured fee exceeds the hard cap. This removes the unbounded admin-controlled fee setting and limits the governance or misconfiguration blast radius. The cap was subsequently tightened in `c1e28ac` from `1 ether` to `0.05 ether`. No bounded step-change or timelock mechanism was added, but the core recommendation to enforce a hard maximum was implemented.

[F-2026-15645](#) - setChainlinkStalePeriod(0) Disables Oracle

Freshness Validation - Info

Description:

The **UniversalGateway** contract allows admin to set `chainlinkStalePeriod` to any value, including `0`:

```
function setChainlinkStalePeriod(uint256 stalePeriodSec) external onlyRole(DE
FAULT_ADMIN_ROLE) whenNotPaused {
    chainlinkStalePeriod = stalePeriodSec;
}
```

In `getEthUsdPrice`, freshness validation is only enforced when `chainlinkStalePeriod != 0`:

```
if (chainlinkStalePeriod != 0 && block.timestamp - updatedAt > chainlinkStale
Period) {
    revert Errors.InvalidData();
}
```

Therefore, setting `chainlinkStalePeriod = 0` fully disables staleness checks and accepts arbitrarily old oracle data.

The protocol may continue operating using stale ETH/USD prices.

USD-based protections (`_checkUSDCaps`, `_checkBlockUSDCap`, `getMinMaxValueForNative`) can become inaccurate.

This can lead to under-enforcement or over-enforcement of caps, depending on market movement during oracle inactivity.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Recommendations

Remediation:

- Enforce a non-zero minimum stale period:
 - e.g., `require(stalePeriodSec >= MIN_STALE_PERIOD && stalePeriodSec <= MAX_STALE_PERIOD)`.
- If disabling is needed for emergencies, use an explicit separate flag/event (e.g., `freshnessCheckDisabled`) with stricter governance controls (timelock/multisig policy).

Resolution:

Fixed in `f92f02e`:

`MIN_CHAINLINK_STALE_PERIOD` was introduced and `setChainlinkStalePeriod` now reverts with `Errors.InvalidInput` when `stalePeriodSec` is below that minimum, including `0`. The current minimum is `10 minutes`, while the initializer default remains `1 hours`. This prevents admin misconfiguration from disabling Chainlink freshness validation through a zero stale period.

F-2026-15682 - Misleading Function Name setRouters - Info

Description:

The function is named `setRouters`, which implies configuring multiple routers, but it actually sets exactly one Uniswap V3 factory and one swap router address.

```
function setRouters(address factory, address router) external onlyRole(DEFAULT_ADMIN_ROLE) whenNotPaused {
    if (factory == address(0) || router == address(0)) revert Errors.ZeroAddress();
    uniV3Factory = IUniswapV3Factory(factory);
    uniV3Router = ISwapRouterV3(router);
}
```

This creates naming ambiguity for integrators and reviewers, suggests multi-router support that does not exist and increases operational risk during admin changes (wrong assumptions about fallback or router sets).

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Rename to a more accurate name, e.g.:

- `setUniswapV3Endpoints`
- `setUniswapV3FactoryAndRouter`
- `setRouterConfig`

Also add a dedicated event to improve observability, e.g.:

```
RoutersUpdated(address oldFactory, address newFactory, address oldRouter, address  
newRouter) .
```

Resolution:

Fixed in `983f879` :

`setRouters` was renamed to `setUniswapV3Config`, accurately reflecting that the function configures one Uniswap V3 factory and one swap router. The update also added `UniswapV3ConfigUpdated(oldFactory, newFactory, oldRouter, newRouter)`, emitted on configuration changes to improve admin-action observability.

F-2026-15693 - Per-Block USD Cap Uses Mixed-Price Accounting Within the Same Block - Info

Description:

`_checkBlockUSDCap` function in the **UniversalGateway** contract enforces the per-block cap by storing the already-converted USD usage in `_consumedUSDinBlock` and adding the USD value of the current transaction.

```
function _checkBlockUSDCap(uint256 amountWei) private {
    uint256 cap = BLOCK_USD_CAP;
    if (cap == 0) return;
    if (block.number != _lastBlockNumber) {
        _lastBlockNumber = block.number;
        _consumedUSDinBlock = 0;
    }
    uint256 usd1e18 = quoteEthAmountInUsd1e18(amountWei);
    if (usd1e18 > cap) revert Errors.BlockCapLimitExceeded();
    unchecked {
        uint256 newUsed = _consumedUSDinBlock + usd1e18;
        if (newUsed > cap) revert Errors.BlockCapLimitExceeded();
        _consumedUSDinBlock = newUsed;
    }
}
```

The issue is that `_consumedUSDinBlock` is not derived from the raw ETH amount deposited in the block. It is derived from historical ETH amounts converted using whatever Chainlink ETH/USD price was returned at the time each prior transaction executed. If the oracle price changes within the same block, the contract mixes:

- old block usage valued at the old price, and
- new block usage valued at the new price.

This means the block cap is not enforced against a single consistent price basis.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate:

2/5

Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Use one of these safer approaches:

- Store raw wei consumed per block, and on each check compute:
 - `(consumedWeiInBlock + amountWei) * currentPrice / 1e18`
- Or snapshot one ETH/USD price per block and use that same price for all transactions within that block.
- Avoid summing USD values that were computed using potentially different oracle rounds.

Resolution: **Fixed in 181afdf :**

`_checkBlockUSDCap` now accumulates raw native wei in `_consumedWeiInBlock` instead of accumulating pre-converted USD values. On each check, the function converts the full cumulative wei amount using the current oracle price and compares that single total against `BLOCK_USD_CAP`. This removes mixed-price accounting within the same block and enforces the block cap against one consistent price basis per check.

F-2026-15725 - Redundant Initializable and ContextUpgradeable Inheritance - Info

Description:

Several contracts explicitly inherit from `Initializable` and/or `ContextUpgradeable`, even though these contracts are already inherited transitively through other OpenZeppelin upgradeable base contracts, such as `AccessControlUpgradeable` and `PausableUpgradeable`.

```
abstract contract AccessControlUpgradeable is Initializable, ContextUpgradeable,
    IAccessControl, ERC165Upgradeable {}

abstract contract PausableUpgradeable is Initializable, ContextUpgradeable {}
```

This results in redundant inheritance, where the same base contracts are included multiple times in the inheritance hierarchy without introducing additional functionality. As a consequence, the inheritance graph becomes more complex and less readable, increasing maintenance overhead and potentially complicating future modifications.

Instances of redundancy include:

- `UniversalGateway.sol` - Redundant inheritance: `Initializable`, `ContextUpgradeable`
- `Vault.sol` - Redundant inheritance: `Initializable`, `ContextUpgradeable`
- `VaultPC.sol` - Redundant inheritance: `Initializable`, `ContextUpgradeable`
- `UniversalGatewayPC.sol` - Redundant inheritance: `Initializable`

Assets:

- `UniversalGateway.sol` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]
- `Vault.sol` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]
- `UniversalGatewayPC.sol` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]
- `VaultPC.sol` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate:

1/5

Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: It is recommended to remove explicitly declared base contracts that are already inherited transitively through other parent contracts, ensuring that only necessary inheritance relationships are retained. This reduces complexity and improves readability without altering behavior.

Resolution: **Fixed in 9ae63d9 :**

Redundant explicit inheritance and unused imports for `Initializable` and `ContextUpgradeable` were removed from `UniversalGateway`, `Vault`, and `VaultPC`; redundant `Initializable` inheritance was removed from `UniversalGatewayPC`. The contracts still receive initializer functionality transitively through the OpenZeppelin upgradeable base contracts, so behavior is preserved while the inheritance graph is simplified.

F-2026-15733 - ETH Permanently Locked in Gateway on ERC20 Revert/Rescue Due to Missing msg.value Validation - Info

Description:

`revertUniversalTx` and `rescueFunds` are `payable` functions restricted to `VAULT_ROLE`. For ERC20 token reverts, the functions validate `msg.value` only for native token path (`token == address(0)`), but perform no check when `token != address(0)`:

```
function _validateRevertParams(bytes32 subTxId, uint256 amount, address token
, address revertRecipient) private {
    if (isExecuted[subTxId]) revert Errors.PayloadExecuted();
    if (revertRecipient == address(0)) revert Errors.InvalidRecipient();
    if (amount == 0 || (token == address(0) && msg.value != amount)) revert Er
rors.InvalidAmount();
    isExecuted[subTxId] = true;
}
```

The condition `(token == address(0) && msg.value != amount)` only validates `msg.value` when the token is native. When `token != address(0)` (ERC20 revert/rescue), any `msg.value` is silently accepted and remains in the **Gateway** contract. The ETH then becomes permanently stuck because there is no admin withdrawal or emergency recovery function for ETH in the **Gateway**.

While the current **Vault** implementation does validate `msg.value == 0` for ERC20 paths:

```
} else {
    if (msg.value != 0) revert Errors.InvalidAmount();
```

this protection resides in the caller contract, not in the **Gateway** itself. The Gateway should independently enforce its own invariants regardless of the caller's implementation, especially since `VAULT_ROLE` may be reassigned to a different contract in the future via `setVault`.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	1/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Add explicit `msg.value` validation for ERC20 paths in `_validateRevertParams`:

```
function _validateRevertParams(bytes32 subTxId, uint256 amount, address token
, address revertRecipient) private {
    if (isExecuted[subTxId]) revert Errors.PayloadExecuted();
    if (revertRecipient == address(0)) revert Errors.InvalidRecipient();
    if (amount == 0) revert Errors.InvalidAmount();
    if (token == address(0)) {
        if (msg.value != amount) revert Errors.InvalidAmount();
    } else {
        if (msg.value != 0) revert Errors.InvalidAmount();
    }
    isExecuted[subTxId] = true;
}
```

Resolution: Fixed in `b85502c`:

`UniversalGateway._validateRevertParams` now validates `msg.value` for both branches. Native revert/rescue requires `msg.value == amount`, while ERC20 revert/rescue explicitly requires `msg.value == 0`. This prevents accidental or malicious native funds from being trapped in the Gateway during ERC20 revert or rescue calls, independent of the current Vault caller's own validation.

[F-2026-15734](#) - Inconsistent State Variable Ordering in UniversalGatewayPC Reduces Upgradeability Clarity - Info

Description: In **UniversalGatewayPC**, state variable declarations are split by the constructor:

```
address public UNIVERSAL_CORE;
/// @custom:oz-upgrades-unsafe-allow constructor
constructor() {
    _disableInitializers();
}
IVaultPC public VAULT_PC;
uint256 public nonce;
```

This layout is not functionally incorrect, but it is inconsistent with common upgradeable-contract structure, where all storage variables are typically grouped together before constructor and function definitions.

For upgradeable contracts, storage layout readability is especially important because auditors and maintainers need to reason carefully about slot ordering and future upgrades. Splitting storage declarations across code sections makes that review harder and increases the chance of mistakes during future modifications.

There is no direct runtime impact in the current version. However, this pattern:

- reduces storage layout clarity,
- makes upgrade reviews more error-prone,
- increases the risk of accidental storage mismanagement in future upgrades.

This is primarily a maintainability and upgrade-safety issue.

Assets:

- UniversalGatewayPC.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Group all state variables together in a single storage section near the top of the contract, before the constructor and function definitions. For example:

```
bytes32 public constant PAUSER_ROLE = keccak256("PAUSER_ROLE");
address public UNIVERSAL_CORE;
IVaultPC public VAULT_PC;
uint256 public nonce;
/// @custom:oz-upgrades-unsafe-allow constructor
constructor() {
    _disableInitializers();
}
```

Resolution: Fixed in 846cf17:

`UniversalGatewayPC` state variables are now grouped together before the constructor. `UNIVERSAL_CORE`, `VAULT_PC`, and `nonce` appear in one contiguous storage section, improving upgrade-review readability while preserving storage slot order and runtime behavior.

[F-2026-15735](#) - Mutable State Variables Are Named Like Constants -

Info

Description:

Several mutable storage variables use **ALL_CAPS** naming, which conventionally implies constant/immutable values in Solidity.

Examples:

- **UniversalGateway:** `TSS_ADDRESS`, `VAULT`, `WETH`, `BLOCK_USD_CAP`, `MIN_CAP_UNIVERSAL_TX_USD`, `MAX_CAP_UNIVERSAL_TX_USD`, `CEA_FACTORY`, `INBOUND_FEE`
- **UniversalGatewayPC:** `UNIVERSAL_CORE`, `VAULT_PC`
- **Vault:** `TSS_ADDRESS`

These variables are changed via admin functions such as `setTSS`, `setVault`, `setBlockUsdCap`, `setProtocolFee`, `setCEAFactory`, `setVaultPC`, etc.

No direct runtime vulnerability, but this creates a high risk of operator/developer misunderstanding:

- reviewers may incorrectly assume values are immutable,
- off-chain tooling/integrations may treat these params as static,
- governance/change-risk can be underestimated during incident response.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]
- Vault.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]
- UniversalGatewayPC.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Rename mutable state variables to standard mutable naming (**camelCase**), for example:

- `TSS_ADDRESS` → `tssAddress`
- `VAULT` → `vault`
- `INBOUND_FEE` → `inboundFee`
- `UNIVERSAL_CORE` → `universalCore`
- `VAULT_PC` → `vaultPc`

Reserve **ALL_CAPS** strictly for constant/immutable symbols.

If renaming is deferred for backward compatibility/readability reasons, document clearly in NatSpec that these values are admin-mutable.

Resolution:

Fixed in `4bcab5c` :

Mutable `ALL_CAPS` storage variables were renamed to standard mutable-style names.

F-2026-15736 - Dead WETH Branch in `_findV3PoolWithNative` - Info

Description:

`_findV3PoolWithNative` contains a branch for `tokenIn == WETH`:

```
if (tokenIn == WETH) {  
    // Caller should handle the WETH fast-path; we return zeroed pool/fee here  
    .  
    return (IUniswapV3Pool(address(0)), 0);  
}
```

However, this branch is unreachable in the current call flow because `swapToNative` handles `tokenIn == WETH` earlier and returns immediately:

```
if (tokenIn == WETH) {  
    IERC20(WETH).safeTransferFrom(_msgSender(), address(this), amountIn);  
    . . .  
    return ethOut;  
}  
(IUniswapV3Pool pool, uint24 fee) = _findV3PoolWithNative(tokenIn);
```

So `_findV3PoolWithNative` is only called when `tokenIn != WETH`, making the internal `WETH` guard dead code.

Dead branches increase maintenance burden and can mislead reviewers into thinking this path is actively used/tested.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Remove the unreachable `tokenIn == WETH` branch from `_findV3PoolWithNative`, or refactor so the function itself is the single source of truth for `WETH` handling (and update call sites accordingly).

Prefer one canonical place for this decision to reduce ambiguity.

Resolution:

Fixed in `e6ba9cb` :

The unreachable `tokenIn == WETH` branch was removed from `_findV3PoolWithNative`. `WETH` handling remains solely in `_swapToNative` through the existing fast path that returns before pool lookup. This is a readability and maintainability cleanup with no behavioral change.

[F-2026-15738](#) - initialize Silently Skips Uniswap Configuration If Only One Address Is Zero - Info

Description:

During initialization of **UniversalGateway**, Uniswap addresses are set only when both factory and router are non-zero. If one is zero and the other is valid, the contract does not revert; it silently leaves both router/factory config unset.

```
if (factory != address(0) && router != address(0)) {  
    uniV3Factory = IUniswapV3Factory(factory);  
    uniV3Router = ISwapRouterV3(router);  
}
```

Deployment can appear successful while swap functionality is effectively disabled. This creates hidden misconfiguration risk and delayed operational failure when token-to-native swap flows are first used.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Add explicit validation:

- revert if exactly one of (`factory` , `router`) is zero,
- allow either both zero (intentionally disabled) or both non-zero (configured).

Resolution:

Fixed in [112e627](#) :

`UniversalGateway.initialize` now rejects half-configured Uniswap inputs. If exactly one of `factory` or `router` is zero, initialization reverts with `Errors.ZeroAddress`. The implementation still permits both addresses to be zero for intentionally disabled swap support, and permits both non-zero addresses for a fully configured deployment.

F-2026-15739 - VaultPC Withdraw Events Omit Recipient Address - Info

Description: `withdraw` and `withdrawToken` emit `FeesWithdrawn(caller, token, amount)` but do not include the to recipient.

```
emit FeesWithdrawn(msg.sender, address(0), amount);
```

Off-chain accounting and monitoring cannot reliably determine where funds were sent from event data alone. This weakens auditability and incident response.

Assets:

- UniversalGatewayPC.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate: 1/5
Likelihood Rate: 2/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

Remediation: Extend event schema to include destination address:

```
event FeesWithdrawn(address indexed caller, address indexed to, address indexed token, uint256 amount);
```

Resolution: Fixed in `c1c0ccb`:

`FeesWithdrawn` now includes an indexed `to` recipient field. Both `withdraw` and `withdrawToken` emit `FeesWithdrawn(msg.sender, to, token, amount)`, allowing off-chain accounting and monitoring to identify the withdrawal recipient directly from event data.

[F-2026-15741](#) - Redundant Deadline Validation in Token-Gas Flow - Info

Description: In the token-gas entrypoint `sendUniversalTx(UniversalTokenTxRequest)`, `deadline` is validated before calling `swapToNative`:

```
if (reqToken.deadline != 0 && reqToken.deadline < block.timestamp) revert Errors.SlippageExceededOrExpired();
```

The same logical validation is performed again inside `swapToNative`:

```
if (deadline == 0) {  
    deadline = block.timestamp + defaultSwapDeadlineSec;  
} else if (deadline < block.timestamp) {  
    revert Errors.SlippageExceededOrExpired();  
}
```

So for non-zero deadlines, the contract performs duplicate checks on the same value in the same call path.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate: 1/5
Likelihood Rate: 1/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

Remediation: Keep deadline validation in one place only (preferably `swapToNative`, where `deadline` is actually consumed).

Then remove the pre-check from `sendUniversalTx(UniversalTokenTxRequest)` to keep logic centralized and reduce drift risk.

Resolution:

Fixed in `c153374`:

The redundant deadline pre-check was removed from `sendUniversalTx(UniversalTokenTxRequest)`. Deadline validation is now centralized in `_swapToNative`, which handles both expired non-zero deadlines and `deadline == 0` default-window substitution. Behavior remains unchanged while duplicate validation and drift risk are removed.

F-2026-15742 - Inconsistent Function Naming vs Visibility

Convention - Info

Description:

UniversalGateway uses inconsistent naming relative to standard Solidity visibility conventions:

- `_checkUSDCaps` is public but prefixed with `_` (typically reserved for internal/private helpers),
- `swapToNative` and `_findV3PoolWithNative` are internal, but only one follows private-helper naming consistently (`_findV3PoolWithNative` does, `swapToNative` does not).

Examples:

```
function _checkUSDCaps(uint256 amount) public view {
    uint256 usdValue = quoteEthAmountInUsd1e18(amount);
    if (usdValue < MIN_CAP_UNIVERSAL_TX_USD) revert Errors.InvalidAmount();
    if (usdValue > MAX_CAP_UNIVERSAL_TX_USD) revert Errors.InvalidAmount();
}

function swapToNative(address tokenIn, uint256 amountIn, uint256 amountOutMin
    ETH, uint256 deadline)
    internal
    returns (uint256 ethOut)
```

Inconsistent naming weakens code readability and can mislead reviewers/integrators about intended API surface and mutability boundaries, especially in security-sensitive codebases.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Apply a consistent convention across the contract:

- public / external functions: no underscore prefix,
- internal / private helpers: underscore prefix.

For example:

- `_checkUSDCaps` → `checkUSDCaps` (if intentionally public), or make it internal and keep underscore,
- `swapToNative` → `_swapToNative` (if it remains internal).

Resolution:

Fixed in `b36e4af`:

Function names were aligned with Solidity visibility conventions. Public `_checkUSDCaps` was renamed to `checkUSDCaps`, and internal `swapToNative` was renamed to `_swapToNative`. Related call sites, interface documentation, tests, and protocol documentation were updated. This was a pure rename with no behavioral change.

F-2026-15743 - Unreachable else Branch in _routeUniversalTx - Info

Description:

`_routeUniversalTx` ends with an `else` statement that reverts with `InvalidTxType`:

```
else if (txType TX_TYPE.FUNDS || txType TX_TYPE.FUNDS_AND_PAYLOAD) {
    _sendTxWithFunds(req, nativeValue, txType, fromCEA);
}
// Route 3: Invalid
else {
    revert Errors.InvalidTxType();
}
```

However, `txType` is always produced by `_fetchTxType`, which already either:

- returns one of the four handled values (`GAS`, `GAS_AND_PAYLOAD`, `FUNDS`, `FUNDS_AND_PAYLOAD`), or
- reverts.

So under the current control flow, this terminal else statement is unreachable.

This is dead defensive code that can slightly reduce clarity during review and static analysis.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Either:

1. remove the unreachable `else` branch to simplify control flow, or
2. keep it as defensive code but document explicitly that it is an invariant guard against future enum/logic drift.

If retained, a short comment like “defensive future-proof guard” is enough.

Resolution:

Fixed in `af82835` :

The terminal `InvalidTxType` branch remains in `_routeUniversalTx`, but it is now explicitly documented as unreachable under the current `_fetchTxType` logic and retained as a defensive invariant guard for future `TX_TYPE` enum additions or routing refactors. The ambiguity for reviewers is reduced, but the dead defensive branch was not removed.

[F-2026-15744](#) - Duplicate Logic in revertUniversalTx and rescueFunds Increases Maintenance Risk - Info

Description: Both **UniversalGateway** and **Vault** implement `revertUniversalTx` and `rescueFunds` with nearly identical logic. In each pair, the functions:

- validate common parameters,
- branch on `token == address(0)` vs **ERC20**,
- perform the same transfer logic,
- differ only in the emitted event name.

In **UniversalGateway**:

```
function revertUniversalTx(...) external payable nonReentrant whenNotPaused onlyRole(VAULT_ROLE) {
    _validateRevertParams(subTxId, amount, token, revertInstruction.revertRecipient);
    if (token == address(0)) {
        (bool ok,) = payable(revertInstruction.revertRecipient).call{ value: amount }("");
        if (!ok) revert Errors.WithdrawFailed();
    } else {
        IERC20(token).safeTransfer(revertInstruction.revertRecipient, amount);
    }
    emit RevertUniversalTx(...);
}

function rescueFunds(...) external payable nonReentrant whenNotPaused onlyRole(VAULT_ROLE) {
    _validateRevertParams(subTxId, amount, token, revertInstruction.revertRecipient);
    if (token == address(0)) {
        (bool ok,) = payable(revertInstruction.revertRecipient).call{ value: amount }("");
        if (!ok) revert Errors.WithdrawFailed();
    } else {
        IERC20(token).safeTransfer(revertInstruction.revertRecipient, amount);
    }
    emit FundsRescued(...);
}
```

The same duplication pattern also exists in **Vault**.

Duplicated critical fund-movement logic creates a maintenance hazard:

- future bug fixes may be applied to one function but not the other,
- security checks can diverge over time.

This is especially undesirable in revert/rescue flows, where consistency is important.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]
- Vault.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Extract the shared transfer/refund logic into a single internal helper and keep only event emission in the public wrappers. For example:

```
function _processRevertOrRescue(address token, uint256 amount, address recipient) internal {
    if (token == address(0)) {
        (bool ok,) = payable(recipient).call{ value: amount }("");
        if (!ok) revert Errors.WithdrawFailed();
    } else {
        IERC20(token).safeTransfer(recipient, amount);
    }
}
```

Then call the helper from both `revertUniversalTx` and `rescueFunds`. This reduces drift risk and makes future changes safer.

Resolution:

Accepted:

The duplicated fund-movement logic in `revertUniversalTx` and `rescueFunds` remains present and is acknowledged as intentional design. The client justified the structure by distinguishing automated revert processing from manual rescue operations, with separate events consumed by different off-chain monitoring pipelines. Given the simple logic and the operational

need for distinct event semantics, the remaining maintenance risk is accepted rather than remediated through helper extraction.

F-2026-15745 - Redundant else if Branch in _sendTxWithFunds - Info

Description:

In `_sendTxWithFunds` from **UniversalGateway**, the `FUNDS_AND_PAYLOAD` logic uses:

```
else if (_req.token != address(0)) {
    uint256 gasAmount = nativeValue;
    _sendTxWithGas(
        TX_TYPE.GAS,
        _msgSender(),
        gasRecipient,
        gasAmount,
        bytes(""),
        _req.revertRecipient,
        _req.signatureData,
        fromCEA
    );
    tokenForFundsAndPayload = _req.token;
}
```

At this point, previous branches already handled:

- `nativeValue == 0`
- `nativeValue > 0 && _req.token == address(0)`

So for the remaining path, `_req.token != address(0)` is always true. This makes `else if (_req.token != address(0))` logically equivalent to a plain else.

Redundant branching:

- reduces readability,
- makes control-flow reasoning harder,
- increases risk of silent logic drift in future refactors.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Replace the redundant condition with `else` and keep a short comment documenting that this is the only remaining valid case. This improves clarity and reduces maintenance risk.

Resolution: **Fixed in `b332183`:**

The redundant `else if (_req.token != address(0))` in `_sendTxWithFunds` Case 2.3 was replaced with a plain `else`. A comment now documents that `_req.token != address(0)` is implied by the preceding branches. This simplifies control flow without changing behavior.

F-2026-15746 - IUniversalGateway.UniversalTxExecuted Is Declared but Never Emitted - Info

Description: The **IUniversalGateway** interface declares an event `UniversalTxExecuted`, suggesting that execution on external chains is signaled by the gateway layer:

```
event UniversalTxExecuted(  
    bytes32 indexed subTxId,  
    bytes32 indexed universalTxId,  
    address indexed pushAccount,  
    address target,  
    address token,  
    uint256 amount,  
    bytes data  
);
```

However, `UniversalGateway` never emits this event. The implementation emits other events (`UniversalTx`, `RevertUniversalTx`, `FundsRescued`, etc.), but not `UniversalTxExecuted`.

This creates ABI/observability inconsistency:

- integrators may rely on a non-existent signal,
- indexers and monitoring systems can miss intended lifecycle milestones,
- interface consumers get misleading expectations about execution telemetry.

Assets:

- UniversalGateway.sol [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway>]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Choose one of the following and keep interface/implementation aligned:

1. Emit `UniversalTxExecuted` from the contract responsible for execution finalization in this architecture, or
2. Remove the event from **IUniversalGateway** if execution signaling is intentionally handled elsewhere (e.g., **Vault/CEA** events).

Also document the canonical execution event source to avoid integration ambiguity.

Resolution:

Fixed in `76bc980` :

The unused `UniversalTxExecuted` event was removed from `IUniversalGateway`, and related documentation references were deleted. The interface no longer suggests that `UniversalGateway` emits an execution-finalization signal. Execution telemetry is represented by `Vault`'s `UniversalTxFinalized` event instead.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/pushchain/push-chain-gateway-contracts
Commit	0928967cce6cdac2a584340892daf621ff173abd
Remediation commit	4249311df6a388265866b1be6de6ac216b3dbfc0
Post Audit Changes	80f7a208bcd484601a86d55e5a3891727e7f0ef9
Whitepaper	-
Requirements	https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway/docs
Technical Requirements	https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway/docs

Asset	Type
Errors.sol [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway]	Smart Contract
Types.sol [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway]	Smart Contract
TypesUG.sol [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway]	Smart Contract
TypesUGPC.sol [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway]	Smart Contract
UniversalGateway.sol [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway]	Smart Contract
UniversalGatewayPC.sol [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway]	Smart Contract
Vault.sol [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway]	Smart Contract
VaultPC.sol [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/evm-gateway]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.